Neuradix Robotics Platform

Product, Functional and Technical Specification

Busuttil Technologies Limited

22 June 2026

Contents

Document status

Field	Value
Product name	**Neuradix**
Formal name	**Neuradix Robotics Platform**
Document	Product, Functional and Technical Specification
Version	0.2 Draft
Date	22 June 2026
Owner	Busuttil Technologies Limited
Intended licence	Apache License 2.0 for the open platform core
Initial reference domain	Autonomous mobile and marine robots
Expansion domain	Space simulation, ground systems, payloads and qualification-oriented flight software
Primary implementation languages	Rust and Python

This document defines the product architecture, sub-platform functions, interfaces, normative requirements, non-functional requirements, developer experience, security model, safety and FDIR model, packaging, interoperability, space/flight profile and phased delivery plan for Neuradix.

The words **MUST**, **MUST NOT**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT** and **MAY** are used as normative requirement terms.

> *Neuradix provides technical mechanisms and evidence generation. It does not by itself confer safety certification, flight qualification, human-rating approval or compliance with a particular mission standard.*

1. Naming and product identity

1.1 Recommended name

The product SHOULD be named simply:

> ***Neuradix***

The formal descriptor is:

> ***Neuradix Robotics Platform***

Recommended tagline:

> ***Dependable autonomy, from simulation to deployment.***

Recommended positioning statement:

> *Neuradix is a Rust-first, contract-driven robotics platform for building autonomous systems that are deterministic where required, observable by default, safe to extend with Python, and reproducible from simulation through field deployment.*

The platform SHOULD NOT adopt a second master brand such as Forge, Fabric or Core. Neuradix is already distinctive, extensible and appropriate for a technology platform. Sub-products should remain descriptive rather than becoming unrelated brands.

1.2 Product family and sub-platform model

Neuradix is one platform with shared contracts and evidence, implemented through foundational services and deployment-oriented sub-platforms.

Foundation services

Foundation	Purpose
Neuradix Contracts	Interface definitions, semantic types, units, frames, clock domains, compatibility rules and code generation
Neuradix Runtime	Component lifecycle, scheduling, supervision, resource accounting and local execution
Neuradix Data Plane	Transport-independent streams, state, commands, tasks, events and queries
Neuradix Safety	Command authority, constraint enforcement, safe-state handling, health supervision and FDIR
Neuradix Record	Recording, indexing, deterministic replay, causal lineage and evidence packaging
Neuradix Security	Component identity, permissions, signing, secure boot integration, audit and update controls

Operational sub-platforms

Sub-platform	Primary responsibility
Neuradix Edge	General robot autonomy, payload processing, perception, estimation, planning and supervisory control on Linux-class computers
Neuradix Embedded	Bounded MCU/RTOS components, local servo/control functions, sensor acquisition and actuator interfaces
Neuradix Flight	Static, deterministic and qualification-oriented spacecraft and launch-vehicle flight software
Neuradix Safety	Independent authority and FDIR service, including deployment as a separate safety island where required
Neuradix Sim	Digital twins, scenario execution, software/processor/hardware-in-the-loop and Monte Carlo verification
Neuradix Ground	Mission control, command validation, telemetry, timelines, operations procedures and evidence capture
Neuradix Studio	Engineering, visualization, graph inspection, debugging, replay, configuration and test authoring
Neuradix Fleet	Multi-robot and constellation inventory, mission coordination, health, staged deployment and update management
Neuradix Bridge	Controlled interoperability with ROS 2, MAVLink, DDS, CAN, serial, industrial and space communication systems
Neuradix Registry	Signed packages, contracts, simulation models, AI models, deployment manifests and provenance metadata

Sub-platform names describe supported execution and operational profiles. They SHALL share the same contract language and evidence model, but they MUST NOT be assumed to have identical timing, security, assurance or certification properties.

1.3 Naming conventions

- Main CLI executable: `neuradix`
- Rust crates: `neuradix-runtime`, `neuradix-sdk`, `neuradix-contracts`, `neuradix-record`
- Python package: `neuradix`
- Environment variables: `NEURADIX_*`
- OCI packages: `registry.neuradix.<tld>/<organisation>/<package>:<version>`
- Contract namespace: `io.neuradix.<domain>.<package>` or an organisation-owned reverse-domain namespace

- Repository organisation: github.com/neuradix if available
- Configuration directory: /etc/neuradix
- Runtime state: /var/lib/neuradix
- Runtime socket/state directory: /run/neuradix

The CLI MAY later acquire a short alias, but the canonical command should remain neuradix to avoid abbreviation conflicts and improve discoverability.

1.4 Recommended domain structure

Assuming the registered domain is neuradix.<tld>:

Address	Purpose
neuradix.<tld>	Product and project home
docs.neuradix.<tld>	Versioned documentation
studio.neuradix.<tld>	Optional hosted Studio or remote access portal
registry.neuradix.<tld>	OCI/package registry
packages.neuradix.<tld>	Searchable package catalogue
community.neuradix.<tld>	Forum, discussions and governance
status.neuradix.<tld>	Hosted service status
security.neuradix.<tld>	Vulnerability disclosure and advisories

A legal trademark search is still required before commercial launch. The naming assessment in this specification is a product recommendation, not legal clearance.

2. Vision

2.1 Product vision

Neuradix will provide a coherent operating platform for dependable autonomous machines. It will combine typed component contracts, bounded execution, transport-independent communications, deterministic simulation, incident replay, safety authority, secure deployment and first-class Rust/Python development.

The platform is not merely a message bus. Its purpose is to make a complete robotic system easier to:

- design;
- understand;
- validate;
- simulate;
- deploy;
- observe;
- diagnose;
- reproduce;
- secure;
- maintain over a long operational life.

2.2 Strategic thesis

Existing robotics frameworks tend to optimise one or more of the following while leaving the rest to integration work:

- middleware flexibility;
- real-time control;

- autonomy behaviours;
- embedded reliability;
- simulation;
- data capture;
- cloud fleet operation;
- AI experimentation.

Neuradix will compete by treating the robot as one governed system with shared contracts and evidence across all of those concerns.

Its differentiator is not merely lower latency. It is **system trustworthiness with reduced integration entropy**.

2.3 Initial market focus and expansion path

The first production profile SHOULD target autonomous mobile and field robots, with an AUV or USV as the primary reference system. The same foundations SHALL support an orderly expansion into space systems.

The intended maturity sequence is:

1. terrestrial and marine simulation, autonomy and operations;
2. space simulation, digital twins and ground systems;
3. non-critical payload and experiment computers;
4. CubeSat and small-spacecraft flight software;
5. launcher supervisory and safety-adjacent functions after sufficient assurance and mission heritage;
6. critical launcher or human-rated functions only under a mission-specific qualification programme.

The platform SHALL clearly distinguish technical capability from mission certification. Installing Neuradix does not itself make a system flight-certified, safety-certified or human-rated.

3. Design principles

Neuradix SHALL be guided by the following principles.

3.1 Contracts before connectivity

A component connection is valid only when its types, units, coordinate frames, timing, authority and security requirements are compatible. The platform should prevent invalid systems before runtime where possible.

3.2 Bounded by default

Queues, memory, execution time, retries and network buffering MUST be bounded or explicitly declared unbounded. Silent unbounded growth is prohibited in production profiles.

3.3 Mixed criticality is explicit

A thruster controller, mission planner, web dashboard and Python object detector do not have equal timing or safety properties. The runtime MUST model those differences.

3.4 Simulation and hardware use the same contracts

Simulated, replayed and physical devices MUST implement the same capability interfaces. Application components should not contain simulation-specific branches.

3.5 Observability is part of correctness

Every significant command and state transition SHOULD be traceable back to its inputs, configuration, component version and safety decision.

3.6 Offline is a normal state

The robot MUST remain safe and operational without cloud or operator connectivity. Networking policies must support disconnection, store-and-forward and constrained links.

3.7 Rust owns trust boundaries

Rust SHOULD implement the runtime, safety paths, drivers, transport, recording and deterministic logic. Python SHOULD be first-class for AI, analysis, mission prototyping and non-critical algorithms, but isolated from safety-critical paths.

3.8 Interoperate without architectural contamination

ROS 2, MAVLink, DDS and other systems are supported through explicit bridges. Their assumptions must not become the internal public API of Neuradix.

3.9 Prefer standards over reinvention

Neuradix SHOULD use established standards for serialization, packaging, observability, recording and plugin interfaces where they fit. New protocols should be introduced only where measurable platform requirements cannot otherwise be met.

3.10 Production is compiled, development is dynamic

Development mode may permit discovery and graph changes. Production deployments SHOULD use a validated, signed and immutable topology.

3.11 Assurance evidence is a platform output

Requirements, contract definitions, generated code, tests, timing measurements, deployment manifests, configuration and mission records SHOULD form one traceable evidence chain. Qualification support is not an after-the-fact documentation exercise.

3.12 Critical profiles minimise the trusted computing base

The general platform MAY offer rich networking, Python, dynamic discovery and web tooling. Embedded safety and flight profiles MUST include only the functions required by their mission and MUST support static, auditable deployment.

4. Scope and non-goals

4.1 In scope

Neuradix includes:

- deployment profiles for Edge, Embedded, Flight, Ground, Safety and Simulation;
- a component model and lifecycle;
- Rust and Python SDKs;
- embedded integration;
- typed data and command primitives;
- transport selection and routing;
- shared-memory large-buffer exchange;
- deterministic and asynchronous executors;
- semantic units, frames and time;
- authority and safety services;
- recording and deterministic replay;
- simulation orchestration;
- deployment manifests and package management;
- security identities and permissions;
- observability and command lineage;
- hardware capability interfaces;

- bridges to major robotics and industrial protocols;
- local developer tools and a graphical Studio;
- optional fleet operation services.

4.2 Explicit non-goals for version 1

Version 1 is not intended to:

- replace Linux or a certified RTOS;
- guarantee hard real-time on an unconfigured general-purpose operating system;
- provide every possible robotics algorithm;
- create a new general-purpose programming language;
- create a new binary container format;
- implement a new internet-scale package registry from first principles;
- make arbitrary Python code safety-critical;
- support every robot class equally from the first release;
- provide safety certification by declaration alone;
- hide networking, timing or data-loss behaviour from developers.

5. Stakeholders and personas

5.1 Robotics systems engineer

Needs to define system topology, interfaces, safety constraints, deployment targets and performance budgets.

5.2 Rust component developer

Builds drivers, control algorithms, transport services, runtime plugins and safety-related components.

5.3 Python AI/research developer

Builds perception, machine-learning inference, scientific analysis and experimental autonomy modules without managing unsafe FFI directly.

5.4 Embedded engineer

Builds MCU firmware, sensor gateways, motor control and safety I/O using `no_std` Rust where practical.

5.5 Simulation engineer

Builds worlds, scenarios, sensor models, hydrodynamics and regression suites.

5.6 Test and safety engineer

Needs evidence, reproducible scenarios, fault injection, requirements traceability and immutable logs.

5.7 Operator/fleet manager

Needs vehicle health, mission status, safe commands, auditability and controlled updates.

5.8 Hardware vendor

Needs stable capability contracts, conformance tests, package signing and a route to certified compatibility.

5.9 Flight software engineer

Needs static topology, bounded execution, deterministic scheduling, typed command and telemetry interfaces, controlled unsafe Rust, target-specific builds, watchdog integration and reproducible qualification evidence.

5.10 Mission operations engineer

Needs authenticated command workflows, timelines, telemetry dictionaries, procedures, event correlation, replay, constrained-link handling and an audit trail of every operational action.

5.11 Safety and product-assurance engineer

Needs requirements traceability, hazard-linked safety rules, test evidence, toolchain records, configuration control, independent review points and clear separation between qualified and non-qualified components.

5.12 Payload and experiment developer

Needs a productive Edge or Embedded environment that can be isolated from critical vehicle control while retaining shared data contracts, simulation models and ground tooling.

6. System architecture

6.1 Architectural overview

Neuradix separates shared platform foundations from deployment-specific execution profiles. Contracts, time semantics, evidence and security policy remain consistent across the platform, while each profile restricts scheduling, dynamic behaviour, language use and connectivity according to its assurance needs.

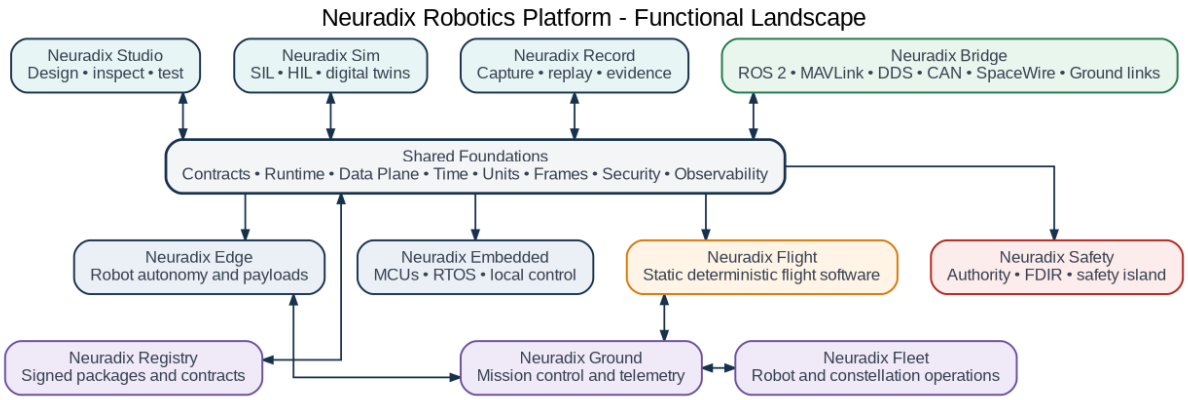

Figure 1 - The Neuradix functional landscape and principal sub-platforms.

6.2 Shared logical layers

The platform is organised into the following logical layers:

Layer	Functions
Engineering and operations	Studio, Ground, Fleet, Registry, mission tooling and CI/CD
Application components	Drivers, estimation, perception, navigation, guidance, control, payload and mission logic
Governed platform services	Lifecycle, configuration, time, frames, safety, FDIR, health, recording, identity and audit
Execution runtime	Deterministic, asynchronous, embedded, Python, WebAssembly and flight-restricted executors
Typed data plane	Stream, State, Command, Task, Event and Query primitives
Transport adaptation	In-process, shared memory, Zenoh, QUIC, CAN, serial, SpaceWire and mission-specific links
Targets	Linux, PREEMPT_RT Linux, RTOS, bare metal, simulation, workstation, cloud and ground infrastructure

6.3 Profile compiler

A topology and policy compiler SHALL transform contracts, component declarations and deployment policy into a profile-specific executable plan.

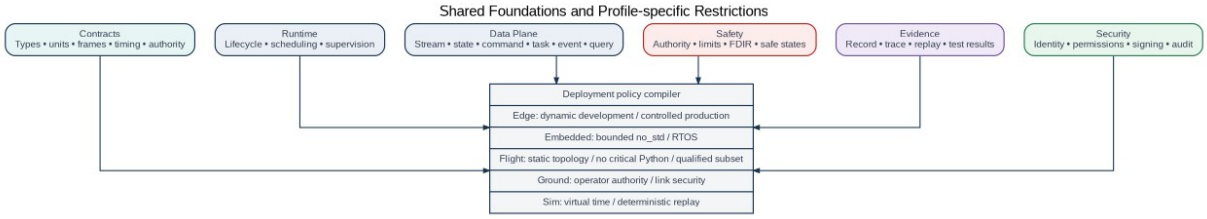

Figure 2 - Shared foundations are constrained by the selected deployment profile.

The compiler MUST validate at least:

- type, schema and version compatibility;
- physical units and coordinate frames;
- clock domains and maximum data age;
- authority and actuator access paths;
- queue and memory bounds;
- component placement and resource capacity;
- execution class and timing budget;
- security permissions and trust domain;
- safety monitoring and fallback paths;
- profile restrictions such as static topology or prohibited language/runtime features.

6.4 Deployment architecture

A deployed Neuradix system consists of:

1. **Components** implementing declared contracts.
2. **Nodes** representing physical or virtual compute targets.
3. **A signed topology manifest** defining placement, connections and policy.
4. **A runtime agent or statically linked flight runtime** on each node.
5. **A resolved data-plane adapter** selected for every connection.
6. **Platform services** for identity, time, health, evidence and safety.
7. **Signed artifacts** containing binaries, Python environments, schemas, models and metadata.
8. **A ground or operator trust domain** where commands and deployments are authorised.

6.5 Architectural rule

Application source code MUST depend on Neuradix contracts and SDK interfaces, not on a specific transport implementation. Components that require direct hardware or transport access SHALL declare that dependency explicitly and SHALL be isolated behind a capability interface.

7. Functional sub-platform architecture

7.1 Cross-platform functional model

Every sub-platform SHALL consume the same canonical contracts and SHALL produce compatible evidence. The functional boundary is defined by deployment policy rather than by creating unrelated frameworks.

Requirement	Function
NRX-PLT-001	The platform SHALL compile one contract definition into Rust, Python, embedded, flight, web and ground representations where supported.
NRX-PLT-002	The platform SHALL preserve message identity, timestamps, units, frame semantics and provenance across sub-platform boundaries.
NRX-PLT-003	Every command crossing a trust or vehicle boundary SHALL be authenticated, authorised, freshness-checked and auditable.
NRX-PLT-004	Every deployment SHALL declare its profile and SHALL be rejected when a component uses prohibited capabilities.
NRX-PLT-005	Simulation, replay and physical drivers SHALL implement the same capability contracts.
NRX-PLT-006	Safety decisions SHALL remain enforceable when Ground, Fleet, Studio or cloud services are unavailable.
NRX-PLT-007	Evidence SHALL identify the exact contract, component, configuration and deployment versions involved in an operation.

7.2 Neuradix Edge

Neuradix Edge is the general Linux-class onboard platform for terrestrial robots, marine vehicles, spacecraft payload computers and high-level autonomy. It supports deterministic Rust components, asynchronous services and isolated Python AI workers.

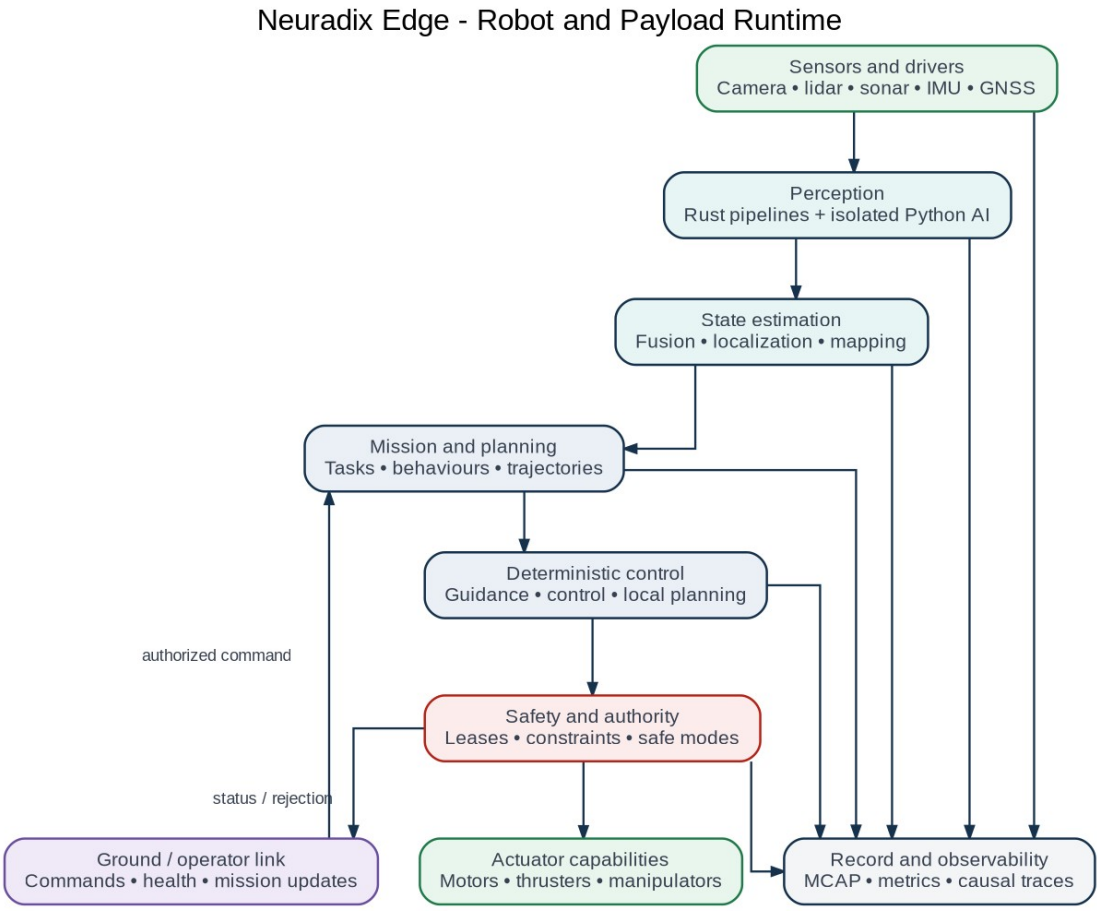

Figure 3 - Neuradix Edge processing and command path.

Requirement	Function
NRX-EDG-001	Edge SHALL execute mixed deterministic, interactive, best-effort and AI workloads in separate supervised executors.
NRX-EDG-002	Python components SHALL run outside critical runtime processes and SHALL have explicit input, output, CPU, memory and restart policy.
NRX-EDG-003	High-bandwidth images, tensors, sonar frames and point clouds SHOULD use loaned or shared-memory buffers locally.
NRX-EDG-004	Edge SHALL continue autonomous operation without Ground or Fleet connectivity.
NRX-EDG-005	Actuator commands SHALL pass through Neuradix Safety or an equivalent approved authority boundary.
NRX-EDG-006	Edge SHALL support shadow components that observe live data without gaining command authority.
NRX-EDG-007	Edge SHALL support deployment on standard and PREEMPT_RT Linux, with explicit disclosure of the guarantee provided by each target.

7.3 Neuradix Embedded

Neuradix Embedded provides bounded components for MCUs, RTOS targets and local controllers. Its normal role is sensor acquisition, preprocessing, servo control, actuator management and hardware safing.

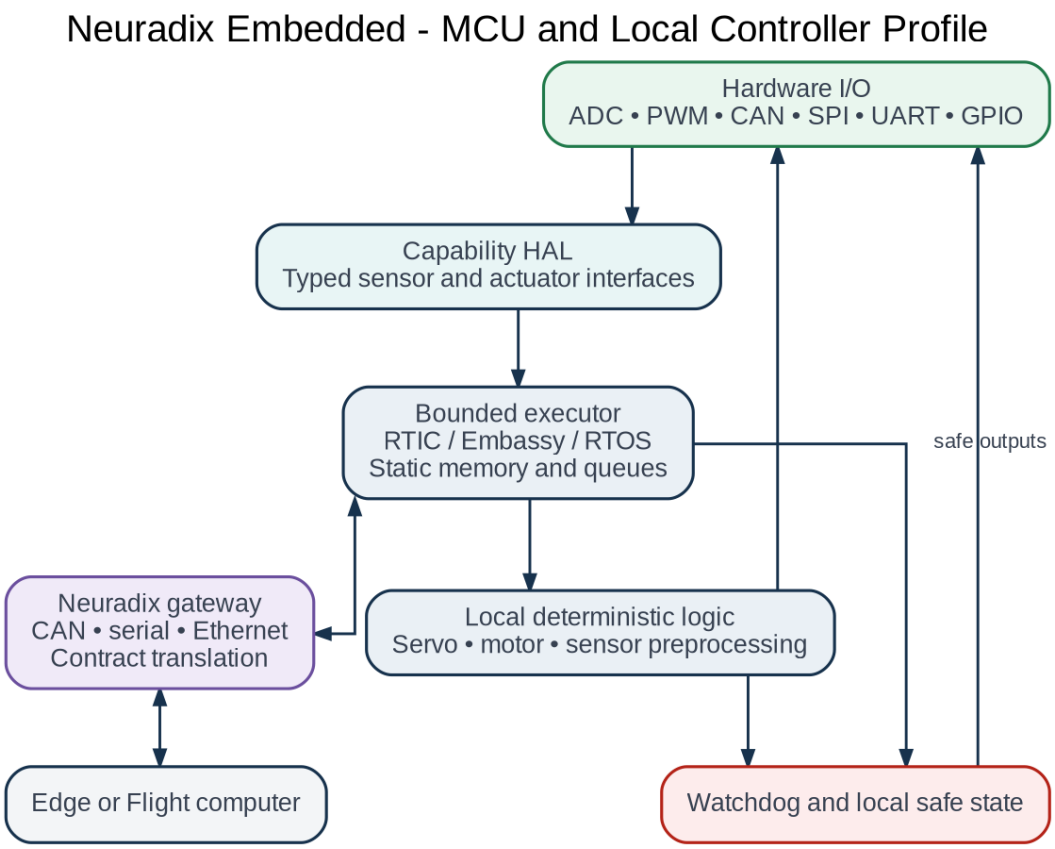

Figure 4 - Neuradix Embedded and its gateway to an Edge or Flight computer.

Requirement	Function
NRX-EMB-001	Embedded SHALL support no_std Rust and static-memory operation for selected targets.
NRX-EMB-002	Queues, tasks, stack budgets and communication buffers SHALL be bounded at build time or startup.
NRX-EMB-003	The profile SHALL support bare metal, RTIC, Embassy and selected RTOS integration through a common capability contract.
NRX-EMB-004	Every actuator controller SHALL define a local safe output that can be applied without the host computer.
NRX-EMB-005	The embedded gateway SHALL validate contract version, integrity, sequence, freshness and authority before applying commands.
NRX-EMB-006	Hardware-specific unsafe code SHALL be isolated, justified and testable.
NRX-EMB-007	Embedded components SHALL expose health, reset reason, watchdog state and resource-watermark telemetry.

7.4 Neuradix Flight

Neuradix Flight is a restricted profile for spacecraft, launch vehicles and other mission-critical embedded systems. It is not the general Edge runtime with a different configuration; it is a smaller statically constructed runtime and assurance package.

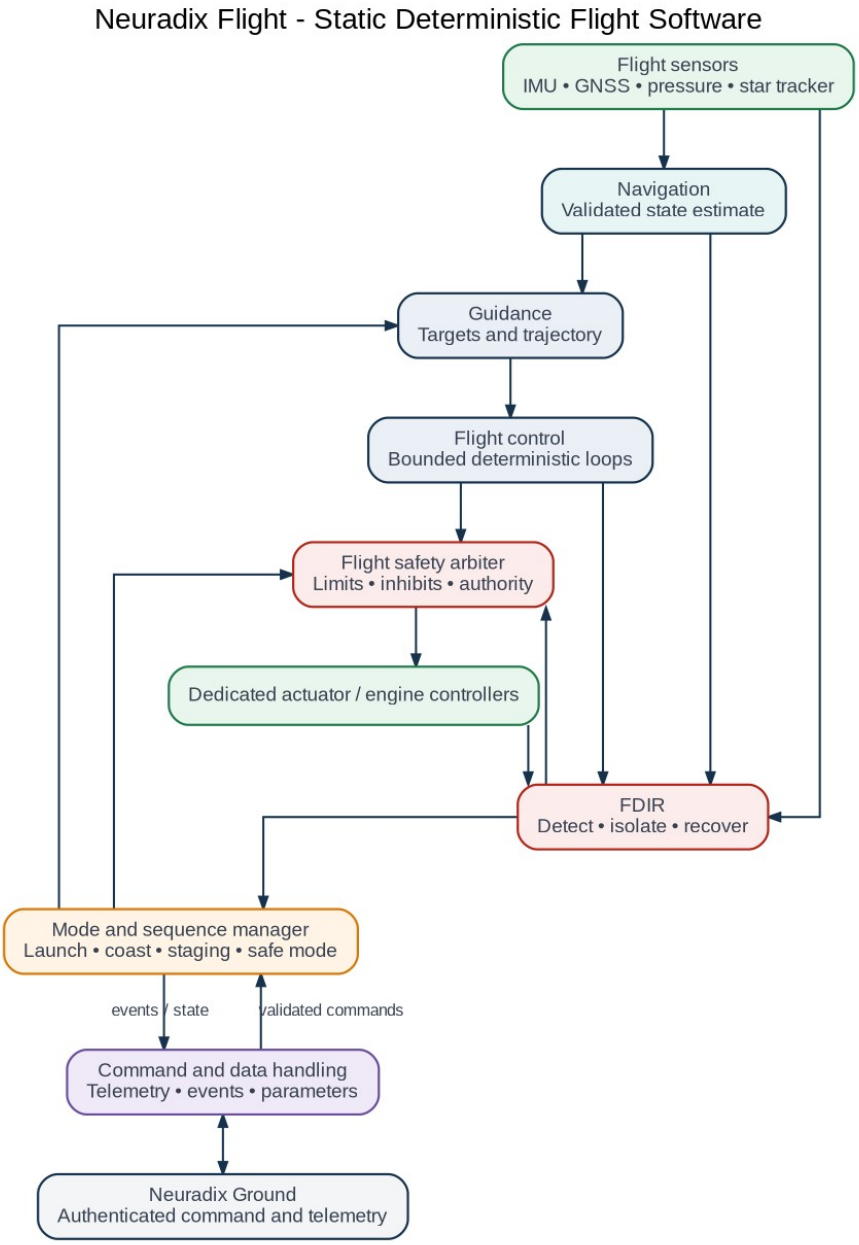

Figure 5 - Neuradix Flight command, control, mode and FDIR functions.

Requirement	Function
NRX-FLT-001	Flight SHALL use a statically compiled component topology with no operational multicast discovery or uncontrolled dynamic loading.
NRX-FLT-002	Flight-critical paths SHALL exclude Python and general-purpose WebAssembly execution.
NRX-FLT-003	All queues, memory pools, retries and execution budgets SHALL be bounded.
NRX-FLT-004	The scheduler SHALL support deterministic periodic, fixed-priority or time-triggered execution selected by mission policy.
NRX-FLT-005	Flight SHALL provide command, telemetry, events, parameters, time, persistent state, watchdog and FDIR services.
NRX-FLT-006	Flight SHALL support target-specific restrictions on Rust language features, dependencies, allocation and unsafe code.
NRX-FLT-007	A mission SHALL be able to separate payload/autonomy processing from vehicle-critical control through hardware and software partitions.

Requirement	Function
NRX-FLT-008	Flight tooling SHALL generate traceability and verification artefacts but SHALL not claim mission certification without the applicable project process.

7.5 Neuradix Safety

Neuradix Safety governs actuator authority and fault response. It may execute as a service within Edge or Flight, as a separate protected process, or on an independent safety computer.

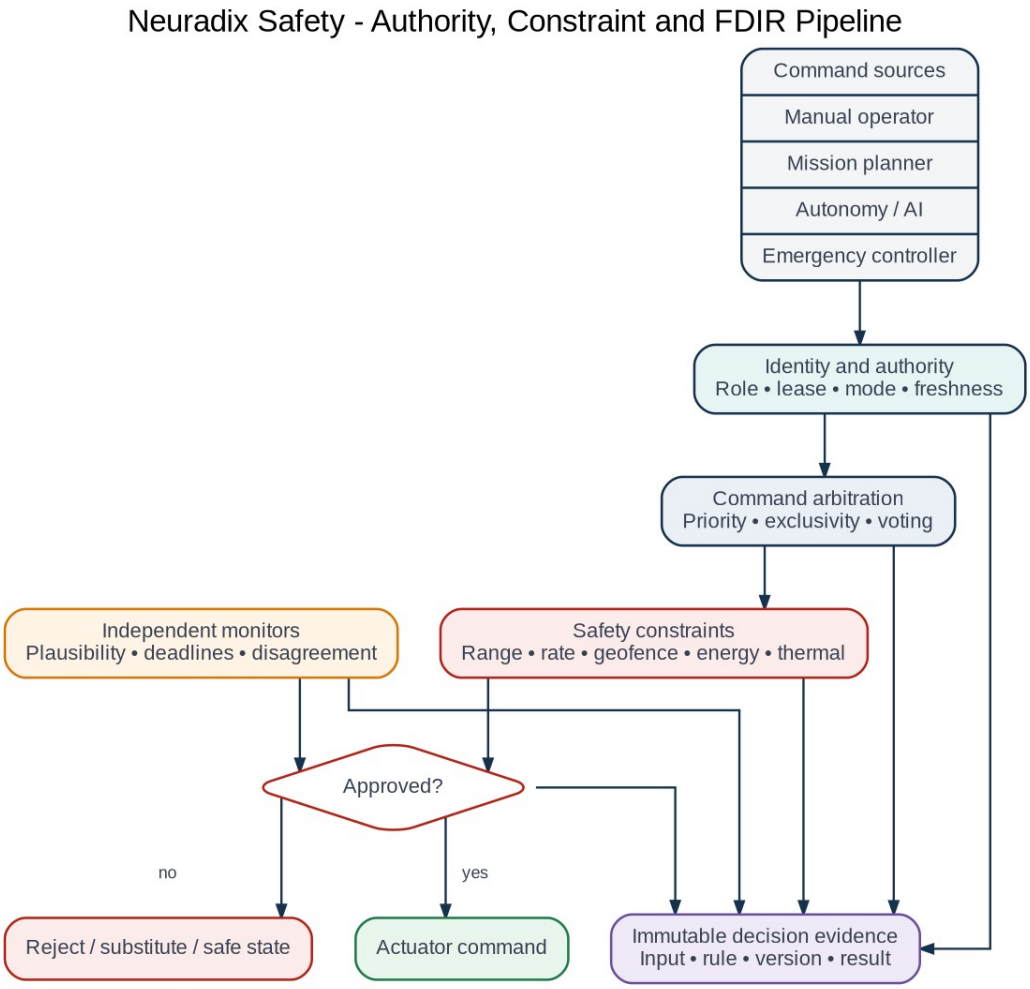

Figure 6 - Command authority, constraints, monitors and decision evidence.

Requirement	Function
NRX-SAF-001	Every command source SHALL possess an identity, role, authority scope and expiry policy.
NRX-SAF-002	Command arbitration SHALL support exclusivity, priority, leases, voting and manual/emergency override.
NRX-SAF-003	Safety constraints SHALL be independently versioned and SHALL identify the rule responsible for modification or rejection.
NRX-SAF-004	FDIR SHALL support detection, isolation, substitution, restart, redundancy switching, degradation and safe-state transition.
NRX-SAF-005	Safety monitors SHALL continue operating when non-critical application components fail.
NRX-SAF-006	Safety output SHALL be fail-silent or fail-safe according to the declared hazard policy.
NRX-SAF-007	Each safety decision SHALL produce immutable causal evidence suitable for incident analysis.

7.6 Neuradix Sim and Neuradix Record

Neuradix Sim executes the actual application graph against virtual capabilities. Neuradix Record preserves sufficient data and configuration to reproduce behaviour.

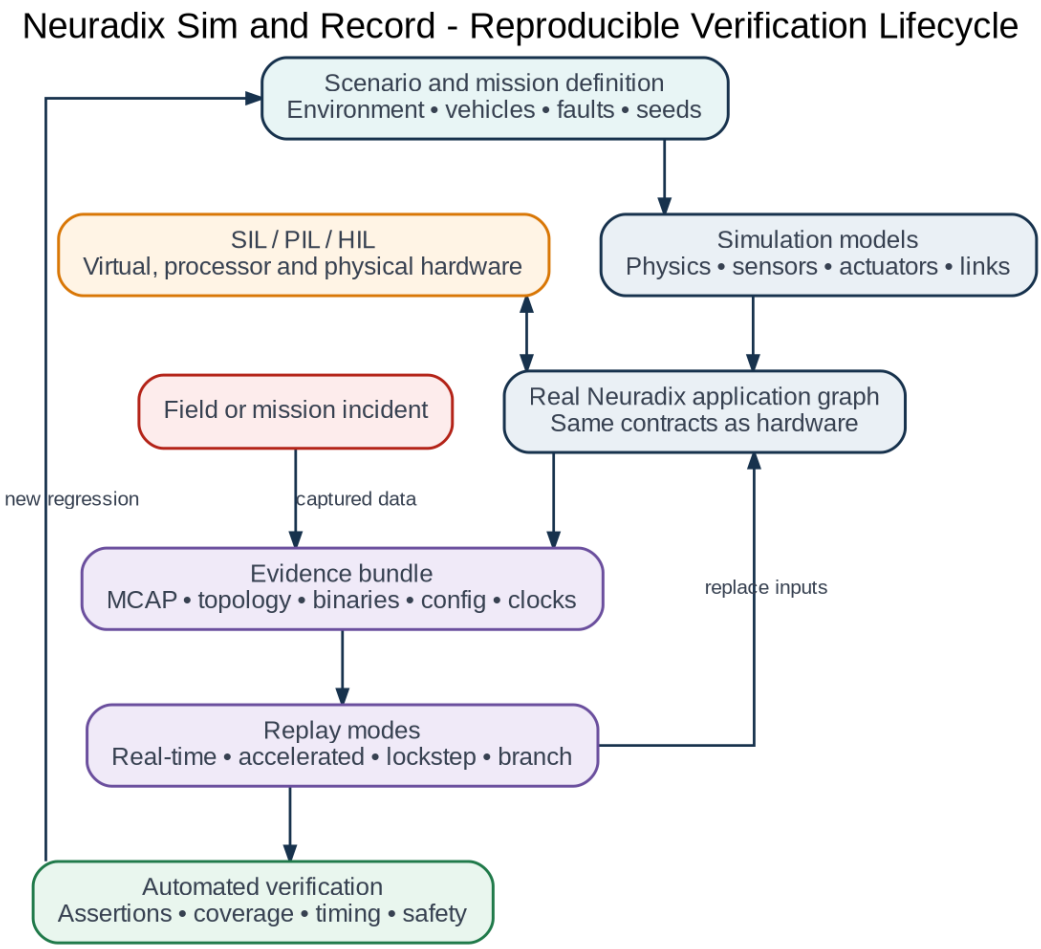

Figure 7 - Scenario execution, evidence capture and regression feedback.

Requirement	Function
NRX-SIM-001	Sim SHALL support virtual, real-time, accelerated and lockstep clock operation.
NRX-SIM-002	Scenarios SHALL declare models, environment, initial conditions, random seeds, faults and assertions.
NRX-SIM-003	The same component binary SHOULD run in simulation and hardware when target architecture permits.
NRX-SIM-004	Sim SHALL support software-in-the-loop, processor-in-the-loop and hardware-in-the-loop topologies.
NRX-REC-001	Record SHALL capture selected streams together with topology, configuration, schemas, clock relations and software hashes.
NRX-REC-002	Replay SHALL support real-time, accelerated, lockstep, branch and counterfactual modes.
NRX-REC-003	A replay run SHALL identify deviations caused by changed code, configuration, timing or nondeterministic input.

7.7 Neuradix Ground, Fleet and Studio

Ground is the mission-operational authority boundary. Fleet coordinates multiple deployed systems. Studio is an engineering tool and MUST NOT silently acquire operational authority merely because it can inspect a live system.

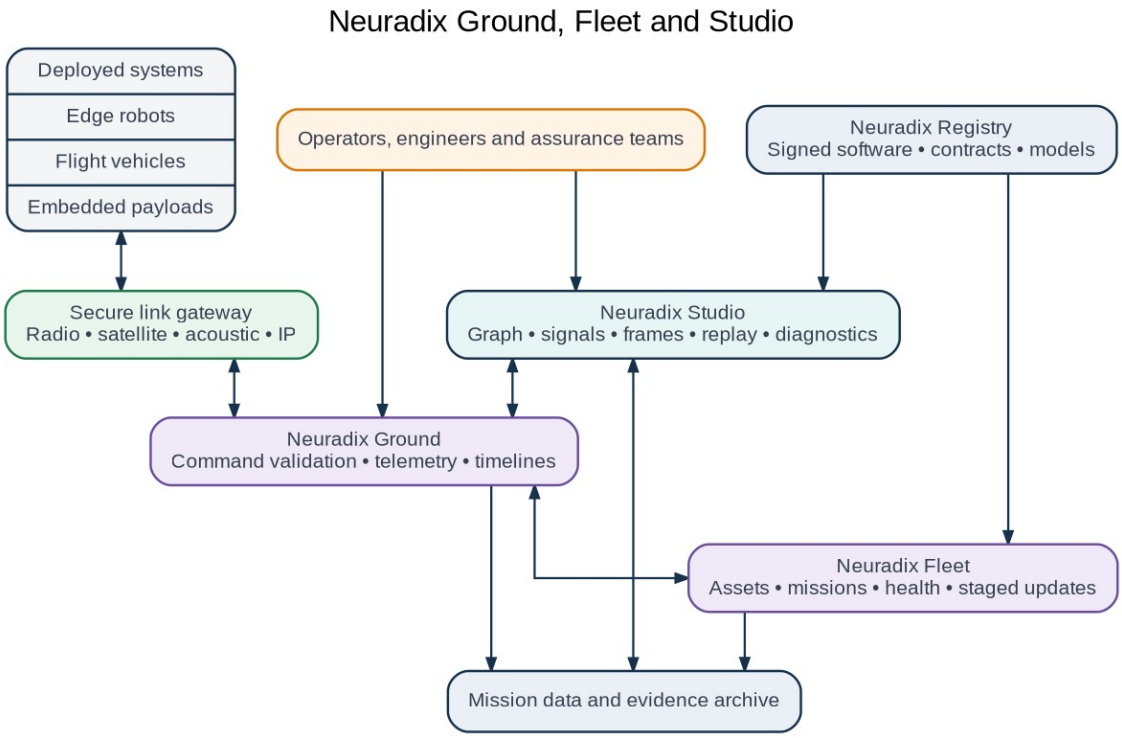

Figure 8 - Operational, engineering, registry and archive interactions.

Requirement	Function
NRX-GND-001	Ground SHALL validate operator identity, command role, vehicle mode, command schema and expiry before uplink.
NRX-GND-002	Ground SHALL maintain command history, acknowledgement, execution result and operator audit records.
NRX-GND-003	Ground SHALL support timelines, procedures, command sequences and link-aware scheduling.
NRX-FLE-001	Fleet SHALL manage asset identity, approved configuration, mission assignment, health and staged updates.
NRX-FLE-002	Fleet SHALL support intermittent connectivity and SHALL never be required for local safety.
NRX-STD-001	Studio SHALL display component graphs, signals, frames, timing, resources, safety decisions and recordings.
NRX-STD-002	Live command capability in Studio SHALL be disabled by default and, when enabled, SHALL use Ground authority services.
NRX-STD-003	Studio SHALL support offline use with local recordings and simulation environments.

7.8 Neuradix Bridge and Registry

Bridge components translate external protocols and frameworks at explicit boundaries. Registry distributes signed artefacts and their provenance.

Requirement	Function
NRX-BRG-001	A bridge SHALL map external names, types, timing and quality semantics into explicit Neuradix contracts.
NRX-BRG-002	Lossy or ambiguous mappings SHALL generate a declared compatibility warning or error.
NRX-BRG-003	External systems SHALL not receive actuator authority unless an explicit policy grants it.

Requirement	Function
NRX-REG-001	Registry SHALL store signed packages, contracts, deployment manifests, models and software bills of materials.
NRX-REG-002	Registry SHALL support immutable versions, revocation, vulnerability advisories and approval channels.
NRX-REG-003	Flight or safety deployments SHALL consume only mission-approved artefacts, not arbitrary latest versions.

7.9 End-to-end lifecycle

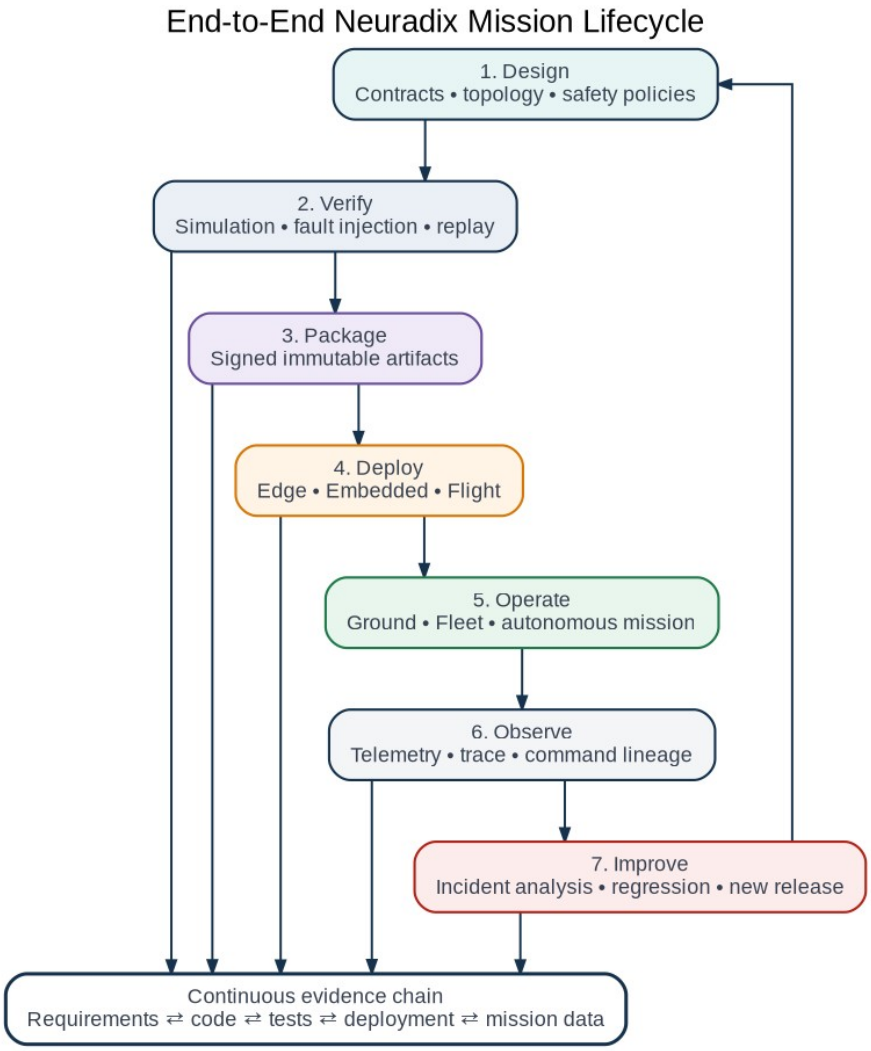

Figure 9 - Continuous design, verification, deployment, operation and improvement.

The platform SHALL preserve an evidence thread from requirements and hazards through contracts, code, tests, deployment and mission data. A field incident SHOULD become a replayable regression case without manually reconstructing the software environment.

7.10 Functional boundary matrix

Onboard and safety profiles

Function	Edge	Embedded	Flight	Safety
General Python components	Isolated	No	No critical use	No
Dynamic discovery	Development only	No	No	No
Static topology	Production option	Required	Required	Required

Function	Edge	Embedded	Flight	Safety
Hard/bounded control	Selected components	Primary	Primary	Primary
Command authority	Requests	Local enforcement	Flight enforcement	Final arbiter
Qualification-oriented evidence	Supported	Supported	Required	Required

Engineering and operations profiles

Function	Sim	Ground	Fleet	Studio
Dynamic topology	Supported	Link dependent	Asset dependent	Development use
Cloud dependency	No	No	Optional	No
Command authority	Simulated	Operator/uplink authority	Delegated coordination	Disabled by default
Deterministic replay	Primary	Analysis and procedure replay	Mission-history analysis	Interactive analysis
Qualification-oriented evidence	Generates	Archives and approves	Tracks deployed baseline	Inspects and exports

7.11 Boundary rules

- Edge autonomy MAY propose actions to Flight, but Flight SHALL validate and retain final authority.
- Ground MAY authorise commands, but an onboard profile SHALL reject commands that are invalid, stale or unsafe.
- Studio MAY inspect any authorised system but SHALL not bypass Ground or Safety command paths.
- Fleet MAY coordinate deployment, but a robot or spacecraft SHALL remain safe when Fleet is unavailable.
- Sim SHALL not introduce simulation-only interfaces into production components.
- Bridge SHALL remain an explicit translation boundary rather than exposing foreign middleware assumptions throughout the platform.

8. Component model

8.1 Component definition

A component is the smallest independently deployable Neuradix application unit. It declares:

- provided and required interfaces;
- inputs and outputs;
- configuration schema;
- lifecycle behaviour;
- execution class;
- resource requirements;
- timing requirements;
- security capabilities;
- health indicators;
- failure policy;
- package identity and version.

8.2 Required lifecycle

Every managed component MUST implement or inherit the following states:

```
Installed -> Configured -> Ready -> Active -> Stopping -> Stopped
                                \-> Faulted -> Recovering -> Ready/Stopped
```

Optional states include `Calibrating`, `Degraded`, `Standby` and `Updating`.

Lifecycle transitions MUST be explicit, observable and auditable. Components MUST NOT begin issuing actuator-affecting commands before reaching `Active` and receiving authority.

8.3 Lifecycle requirements

ID	Requirement
COMP-001	The runtime MUST expose the current lifecycle state of every component.
COMP-002	Lifecycle transitions MUST include reason, initiator, timestamp and result.
COMP-003	Components MUST declare restart policy and maximum restart frequency.
COMP-004	The supervisor MUST detect crash loops and quarantine the component.
COMP-005	A component MAY expose readiness and liveness independently.
COMP-006	Production activation MUST fail if required contracts are unresolved.
COMP-007	Components MUST be addressable by stable logical identity independent of process ID.

8.4 Component isolation classes

Class	Isolation	Typical use
In-process Rust	Shared process, typed references	tightly coupled deterministic pipelines
Native process	OS process boundary	drivers, planners, ordinary services
Python worker	dedicated managed process	AI, scientific code, rapid development
WebAssembly component	capability sandbox	third-party or portable extensions
Embedded endpoint	MCU/RTOS image	motor control, sensor acquisition, safety I/O
External bridge	protocol boundary	ROS 2, MAVLink, OPC UA, legacy system

9. Communication primitives

Neuradix SHALL provide six application-level communication primitives.

9.1 Stream

A Stream represents a sequence of timestamped samples or frames.

Examples:

- IMU samples;
- camera frames;
- sonar frames;
- estimated pose;
- motor feedback.

Streams support policies for reliability, queueing, rate, history, expiry, priority, compression and persistence.

9.2 State

State represents the latest authoritative value with revision and provenance.

Examples:

- vehicle mode;
- active mission;

- battery health;
- selected navigation source;
- safety status.

State updates MUST use revision identifiers. Consumers MAY perform conditional updates to prevent lost updates.

9.3 Command

A Command is a bounded, short-duration request that returns an acknowledgement or result.

Commands MUST support:

- unique command identifier;
- idempotency key;
- deadline and expiry;
- caller identity;
- authorization result;
- acknowledgement state;
- explicit rejection reason.

Examples include `arm`, `set_mode`, `reset_sensor` and `set_light_level`.

9.4 Task

A Task is a long-running, cancellable operation with progress and terminal result.

Tasks MUST support:

- accepted/rejected status;
- progress updates;
- cancellation;
- pause/resume where implemented;
- deadline;
- result or fault;
- persistent task identity.

Examples include `execute_mission`, `calibrate_imu`, `build_map` and `upload_dataset`.

9.5 Event

An Event is an immutable occurrence.

Examples:

- fault detected;
- operator login;
- obstacle observed;
- mode changed;
- safety constraint applied.

Events MUST include origin, event time, receive time, sequence, severity and trace context.

9.6 Query

A Query obtains current, historical or computed information from one or more providers.

Examples:

- retrieve health of all propulsion components;
- fetch map tiles for a region;
- obtain recorded samples within a time range;
- request the latest valid calibration.

Queries MUST define timeout, result cardinality and consistency expectation.

10. Contract system

10.1 Contract contents

Each port or interface contract MUST specify, as applicable:

- canonical type;
- schema version;
- semantic meaning;
- physical unit;
- coordinate frame;
- clock domain;
- timestamp semantics;
- validity duration;
- uncertainty representation;
- expected rate;
- latency and deadline;
- reliability policy;
- queue capacity and overflow behaviour;
- confidentiality/integrity classification;
- authority requirement;
- compatibility rules.

10.2 Canonical schema approach

Version 1 SHOULD use a declarative contract manifest plus established encodings rather than inventing a new binary serialization.

Recommended structure:

- YAML or TOML for human-authored contract metadata;
- Protocol Buffers for ordinary structured wire data;
- typed shared-memory descriptors for large immutable buffers;
- generated Rust and Python bindings;
- JSON projection for web tooling and diagnostics;
- WIT adapters for sandboxed components.

10.3 Example contract

```yaml
apiVersion: neuradix.io/v1alpha1
kind: StreamContract
metadata:
  name: vehicle-depth
  namespace: io.neuradix.navigation
spec:
  payload:
    schema: io.neuradix.navigation.DepthMeasurement@1
  semantics:
    quantity: depth
    unit: metre
    positiveDirection: down
    frame: earth/ned
```

```
time:
  primaryTimestamp: measurement_time
  clockDomain: synchronized_monotonic
  maximumAge: 100ms
delivery:
  reliability: best_effort
  history: latest
  queueCapacity: 4
  overflow: drop_oldest
  expectedRate: 20Hz
quality:
  uncertainty: covariance
  invalidPolicy: reject
```

10.4 Compatibility rules

- Schemas MUST use semantic versioning.
- Patch versions MUST be wire compatible.
- Minor versions MAY add optional fields and compatible enum values.
- Major versions MAY break compatibility and require an explicit adapter.
- Unit conversion MAY be generated only when conversion is unambiguous.
- Frame conversion MUST require an available transform with valid time interval.
- Timing requirements MUST NOT be silently relaxed by an adapter.
- Lossy conversions MUST be explicit and visible in the graph.

10.5 Contract compiler

The `neuradix contract` tool MUST:

- validate contract syntax;
- resolve imports and versions;
- generate Rust and Python types;
- generate Protobuf schemas where applicable;
- produce JSON Schema for configuration and web tooling;
- detect incompatible units and frames;
- generate conformance tests;
- produce machine-readable compatibility reports;
- emit a content-addressed schema identifier.

11. Data model and metadata

11.1 Common envelope

All cross-process messages SHOULD support a common logical envelope containing:

- schema identifier and version;
- source component identity;
- source instance identity;
- sequence number;
- measurement timestamp;
- receive timestamp;
- publish timestamp;
- clock domain;

- trace and correlation identifiers;
- data validity interval;
- quality/uncertainty indicator;
- security label;
- optional frame identifier;
- optional calibration reference.

The envelope MAY be represented out-of-band for zero-copy transports.

11.2 Large data types

The platform SHALL define standard large-buffer types for:

- images;
- encoded video;
- point clouds;
- tensors;
- sonar frames;
- audio;
- occupancy grids;
- meshes;
- map tiles.

A buffer descriptor MUST include:

- element type;
- dimensions and strides;
- encoding;
- byte order;
- memory location;
- ownership/lease state;
- immutability state;
- checksum where required;
- associated frame and timestamp.

11.3 Buffer ownership

- Buffers SHOULD be immutable after publication.
- Local zero-copy consumers receive a bounded lease.
- A producer MUST be prevented from reusing memory while valid consumer leases exist.
- Slow consumers MUST not indefinitely block a critical producer.
- The contract MUST specify whether consumers may be skipped, sampled or copied.

12. Execution and scheduling

12.1 Execution classes

Class	Guarantee intent	Typical workloads
Hard real-time	externally validated bounded deadline	MCU motor control, emergency I/O
Deterministic	bounded queues and controlled scheduling	control, estimation, sensor fusion
Interactive	responsive soft deadlines	drivers, mission logic, operator commands
Best effort	no control-path guarantee	UI, logging exports, maintenance tasks
Batch/AI	throughput and accelerator oriented	inference, mapping, training preparation

12.2 Executor architecture

Neuradix SHOULD provide:

- a deterministic executor for periodic and event-triggered Rust components;
- a fixed-priority executor for configured real-time Linux deployments;
- a Tokio-based asynchronous executor for general I/O;
- dedicated workers for CPU-intensive tasks;
- GPU worker queues;
- managed Python processes;
- embedded adapters for RTIC and Embassy;
- an execution simulator for timing analysis.

The deterministic executor MUST NOT rely on an unconstrained shared async task pool.

12.3 Scheduling declaration

```yaml
execution:
  class: deterministic
  trigger:
    period: 10ms
  deadline: 3ms
  priority: 80
  cpuAffinity: [2]
  memory:
    heapPolicy: bounded
    maximumBytes: 8388608
  blockingCalls: forbidden
  overloadPolicy: enter_degraded_mode
```

12.4 Queue requirements

ID	Requirement
EXEC-001	Every queue MUST have a declared capacity in production.
EXEC-002	Every queue MUST declare overflow behaviour.
EXEC-003	Runtime metrics MUST expose occupancy, drops and latency.
EXEC-004	Blocking publication into a safety-critical path MUST be prohibited unless statically justified.
EXEC-005	Deadline misses MUST generate structured events.
EXEC-006	Overload handling MUST be deterministic and configured.
EXEC-007	Python components MUST NOT execute in hard real-time or deterministic control executors.

12.5 Linux real-time profile

A supported real-time Linux profile SHOULD define:

- tested kernel and PREEMPT_RT versions;
- CPU isolation and affinity;
- interrupt routing;
- memory locking;
- scheduler policy;
- frequency governor;
- network tuning;

- watchdog setup;
- benchmark and acceptance procedures.

The platform MUST clearly distinguish configured real-time capability from ordinary Linux operation.

13. Transport-independent data plane

13.1 Transport selection

The runtime selects or validates a transport according to component placement and contract requirements.

Placement	Preferred transport
Same process	direct typed call or lock-free channel
Same host, small payload	local IPC
Same host, large payload	shared memory
Local network	Zenoh transport profile
Routed/WAN network	Zenoh router or QUIC profile
MCU/sensor bus	CAN, serial, Ethernet or custom adapter
Acoustic/constrained link	store-and-forward gateway

13.2 Initial network backend

Zenoh SHOULD be the first network backend because it provides publish/subscribe, queryable data and routing suitable for edge-to-cloud deployments, while being implemented in Rust. The Neuradix public API MUST nevertheless remain backend-neutral.

13.3 Local shared memory

Neuradix MUST provide a local shared-memory transport with:

- content descriptors;
- lease-based ownership;
- crash recovery;
- bounded pools;
- integrity checks;
- NUMA awareness where relevant;
- metrics for allocation pressure;
- fallback-to-copy policy where configured.

13.4 Data-plane policy

Each connection MAY define:

- reliable or best-effort delivery;
- ordered or latest-only semantics;
- queue capacity;
- drop-oldest, drop-newest, reject, sample or block overflow;
- deadline;
- lifespan;
- priority;
- compression;
- encryption requirement;
- local persistence;
- bandwidth ceiling;

- disconnection behaviour.

14. Time architecture

14.1 Clock domains

The platform MUST distinguish:

- monotonic execution time;
- UTC/wall time;
- sensor hardware time;
- synchronized monotonic time;
- simulation time;
- replay time;
- external navigation time such as GNSS time.

A timestamp without a clock-domain identifier MUST NOT cross a component boundary in production profiles.

14.2 Timestamp semantics

Messages MAY carry:

- measurement time;
- hardware capture time;
- receive time;
- publication time;
- processing completion time.

The contract MUST identify which timestamp is authoritative.

14.3 Synchronization

The time service SHOULD expose:

- synchronization source;
- estimated offset;
- uncertainty;
- last synchronization time;
- holdover state;
- clock-step events;
- quality grade.

14.4 Simulation and replay clock

Simulation and replay MUST support:

- pause;
- step;
- reset;
- accelerated execution;
- lockstep execution;
- deterministic ordering;
- controlled time jumps;
- multiple synchronized simulation participants.

15. Units, frames and spatial semantics

15.1 Units

Physical quantities MUST have explicit canonical units in contracts. Rust SDKs SHOULD provide compile-time unit wrappers where practical. Python SDKs SHOULD expose unit metadata and optional runtime validation without forcing high overhead in numerical inner loops.

15.2 Coordinate frames

Frames MUST have:

- stable identifier;
- owner;
- parent relationship or transform source;
- handedness and axis convention;
- validity interval;
- calibration/version reference;
- uncertainty where available.

15.3 Transform service

The transform service MUST support:

- static transforms;
- dynamic transforms;
- time-indexed lookup;
- interpolation policy;
- transform uncertainty;
- graph validation;
- duplicate authority detection;
- historical replay;
- versioned calibration.

15.4 Marine reference frames

The reference marine profile SHALL support:

- NED and ENU;
- WGS84/geodetic coordinates;
- local tangent frames;
- vehicle body frame;
- sensor frames;
- pressure-derived depth;
- water-relative velocity;
- ground-relative velocity;
- DVL beam frames;
- acoustic transducer frames.

16. Safety and authority architecture

16.1 Safety objective

No ordinary component may directly and unconditionally control a safety-relevant actuator. Actuator requests MUST pass through an authority and safety path appropriate to the robot profile.

16.2 Command path

```
Planner/Operator/Controller
        |
        v
 Authority Manager
        |
        v
 Constraint Evaluator
        |
        v
Rate/Range/Slew Limiter
        |
        v
 Actuator Capability
        |
        v
Hardware Safety Layer
```

16.3 Authority leases

Authority SHALL be granted using time-bounded leases.

A lease includes:

- holder identity;
- controlled capability;
- priority;
- issue and expiry time;
- permitted command envelope;
- pre-emption policy;
- renewal policy;
- reason and issuer.

Expired authority MUST result in a defined safe action.

16.4 Safety constraints

The constraint engine SHOULD support:

- command bounds;
- rate and slew limits;
- geofences;
- depth/altitude limits;
- collision constraints;
- battery and thermal limits;
- actuator health limitations;
- operator override;
- mission phase restrictions;
- communication-loss policy;
- vehicle-specific safe states.

16.5 Fault containment

The runtime MUST support:

- watchdogs;

- deadline monitors;
- liveness and readiness;
- restart budgets;
- circuit breakers;
- component quarantine;
- degraded modes;
- redundant sensor voting;
- plausibility checks;
- emergency stop integration;
- independent hardware safety paths.

16.6 Safety case support

Neuradix SHOULD generate evidence useful to a safety case:

- signed software bill of materials;
- component and schema versions;
- deployment manifest hash;
- test results;
- traceable safety requirements;
- fault-injection evidence;
- configuration history;
- command and intervention audit trail.

Neuradix MUST NOT claim safety certification without the required process, evidence and target-specific assessment.

16.7 Independent safety island

The Safety service SHALL be deployable on a separate processor or protected partition with independent power, watchdog, input monitoring and safe-output paths. The main application may request actions, but SHALL not be able to disable mandatory safety monitors without an authorised mode transition.

16.8 FDIR state model

Fault handling SHALL distinguish detection, confirmation, isolation, accommodation, recovery and return-to-service. Policies SHALL define escalation thresholds and SHALL prevent restart storms or repeated unsafe recovery attempts.

17. Configuration and state management

17.1 Typed configuration

Every component configuration MUST have a versioned schema. The platform MUST validate configuration before activation.

Configuration values SHOULD support:

- type;
- unit;
- allowed range;
- default;
- mutability;
- secrecy classification;
- restart requirement;
- provenance;
- description.

17.2 Configuration classes

- **Build-time:** compiled capability or feature.
- **Deployment-time:** immutable for one deployment.
- **Activation-time:** set before component activation.
- **Runtime mutable:** safely changeable while active.
- **Secret:** delivered through a protected secret provider.

17.3 Transactional updates

Multi-component configuration changes SHOULD be applied transactionally:

1. validate all proposed values;
2. ask affected components to prepare;
3. commit at a defined time or rollback;
4. record the change as an event;
5. retain the previous known-good configuration.

18. Rust SDK

18.1 Goals

The Rust SDK MUST provide:

- ergonomic component definition;
- generated strongly typed ports;
- async and deterministic APIs;
- no unsafe code required for ordinary components;
- `no_std` subsets for embedded profiles;
- structured errors;
- integrated tracing and metrics;
- test harnesses;
- simulation/replay adapters.

18.2 Example Rust component

```rust
use neuradix::prelude::*;

#[component]
#[execution(class = "deterministic", period = "10ms", deadline = "3ms")]
pub struct DepthController {
    #[input(contract = "io.neuradix.navigation/vehicle-depth@1")]
    depth: StreamReader<DepthMeasurement>,

    #[input(contract = "io.neuradix.mission/depth-setpoint@1")]
    setpoint: StateReader<DepthSetpoint>,

    #[output(contract = "io.neuradix.control/vertical-thrust-request@1")]
    thrust: StreamWriter<ThrustRequest>,

    controller: PidController,
}

#[component_impl]
impl DepthController {
    async fn tick(&mut self, cx: &mut TickContext<'_>) -> Result<()> {
```

```
        let depth = self.depth.latest_valid(cx.now())?;
        let setpoint = self.setpoint.get()?;
        let request = self.controller.update(depth, setpoint, cx.dt());
        self.thrust.publish(request).await?;
        Ok(())
    }
}
```

18.3 Rust safety profiles

The SDK SHOULD define lint/configuration profiles:

- ordinary;
- deterministic;
- safety-related;
- embedded `no_std`.

Restricted profiles MAY prohibit:

- unbounded allocation;
- blocking system calls;
- uncontrolled threads;
- network access outside declared ports;
- panic across component boundary;
- unsafe code without explicit review annotation.

19. Python SDK

19.1 Goals

The Python SDK MUST feel native to Python while preserving Neuradix contracts and supervision.

It MUST provide:

- generated type hints;
- async component APIs;
- NumPy-compatible zero-copy views where safe;
- structured configuration;
- tracing and metrics integration;
- managed lifecycle;
- explicit process isolation;
- clear performance diagnostics.

19.2 Integration foundation

The Python package SHOULD use PyO3 and Maturin to expose the Rust client/runtime boundary as normal Python wheels.

19.3 Example Python component

```
from neuradix import Component, StreamInput, StreamOutput, component
from neuradix.types import ImageRGB, Detections

@component(execution="ai", gpu_memory="2GiB")
class ObjectDetector(Component):
    camera: StreamInput[ImageRGB]
    detections: StreamOutput[Detections]
```

```
async def on_camera(self, image: ImageRGB) -> None:
    array = image.numpy(readonly=True)
    result = self.model(array)
    await self.detections.publish(result)
```

19.4 Python restrictions

- Python components MUST run outside hard real-time and deterministic executors.
- Python process crashes MUST be isolated from the core runtime.
- GPU and memory usage SHOULD be constrained by the supervisor.
- Python dependency environments MUST be locked and content-addressed.
- Python components MUST declare whether input samples may be skipped.
- The graph compiler MUST detect Python in a declared deterministic control path.

20. Embedded and microcontroller profile

20.1 Goals

The embedded profile provides compatible contracts and lifecycle semantics on constrained devices without requiring the full Linux runtime.

20.2 Runtime options

- RTIC SHOULD be supported for interrupt-driven deterministic Cortex-M systems.
- Embassy SHOULD be supported for async embedded drivers and services.
- A minimal Neuradix endpoint MUST operate without heap allocation where the selected profile requires it.

20.3 Embedded capabilities

The embedded SDK SHOULD support:

- static contract bindings;
- bounded queues;
- CAN and serial transports;
- time synchronization;
- health and watchdog messages;
- firmware identity;
- secure boot/update hooks;
- compact telemetry profiles;
- local safety state machines.

20.4 MCU gateway

A Linux or embedded gateway MAY proxy a constrained endpoint into the full Neuradix graph while preserving source identity and timestamps.

21. Space and flight profile

21.1 Purpose and applicability

Neuradix Flight extends the platform to spacecraft, launch vehicles, sounding rockets, high-altitude systems and safety-critical mission computers. Its immediate uses are simulation, ground systems, payload software and experimental flight systems. Vehicle-critical deployment requires mission-specific assurance, qualification and organisational approval.

The profile SHALL support the following maturity classes:

Class	Intended use
F0 - Simulation	Desktop, distributed and HIL mission simulation; no flight claim
F1 - Experimental payload	Isolated payload or experiment computer unable to compromise vehicle safety
F2 - Small spacecraft	CubeSat or small-satellite command/data handling, payload coordination and bounded autonomy
F3 - Supervisory flight	Mode management, navigation/guidance supervision, telemetry and redundancy management
F4 - Critical control	Engine, thrust-vector, staging, abort or equivalent critical functions under a dedicated qualification programme
F5 - Human-rated	Out of scope until an independent human-rating and organisational assurance programme exists

21.2 Reference deployment and fault containment

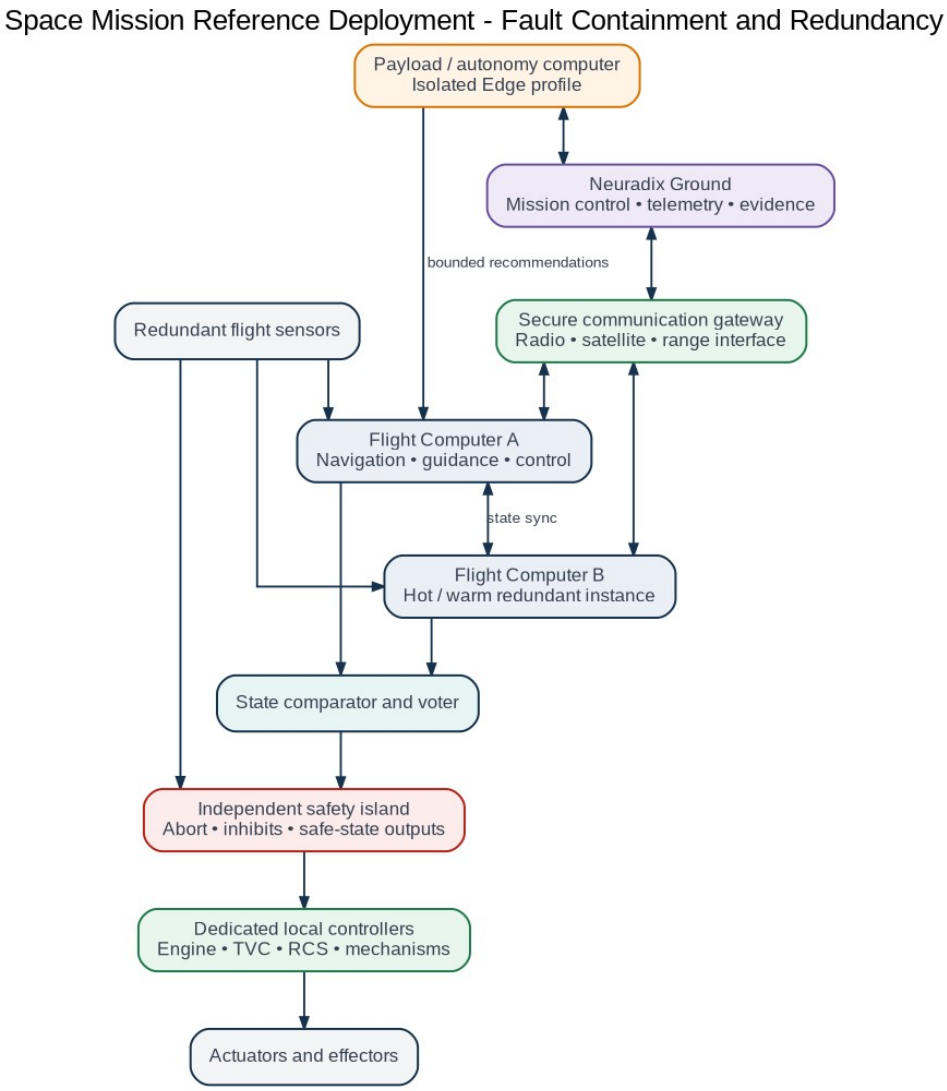

Figure 10 - A reference space deployment with redundant flight computers, an independent safety island and isolated payload autonomy.

The reference architecture separates:

- redundant main flight computers;
- independent safety and abort logic;
- local engine, actuator or mechanism controllers;
- isolated payload/autonomy computing;
- redundant sensors and communication paths;
- authenticated Ground operations.

Neuradix SHALL support this separation but SHALL NOT require every mission to use the same redundancy pattern.

21.3 Restricted flight runtime

The flight runtime MUST minimise its trusted computing base. A mission configuration SHALL be able to prohibit:

- heap allocation after initialization;
- unwinding and uncontrolled panic handling;
- dynamic component loading;
- operational multicast discovery;
- runtime code generation;
- unrestricted recursion;
- unbounded collections and queues;

- unapproved dependencies;
- unreviewed unsafe Rust;
- Python in critical partitions;
- general shell, package-manager or web-server access.

The runtime SHOULD support `no_std` or a controlled target runtime, depending on the selected operating system and hardware.

21.4 Scheduling and timing

The Flight profile SHALL support at least:

- cyclic executive scheduling;
- fixed-priority periodic tasks;
- event-driven tasks with bounded activation and execution;
- rate-group scheduling;
- partitioned execution for mixed-criticality systems;
- deadline and worst-case-execution-time monitoring;
- monotonic, mission-elapsed and synchronized spacecraft time;
- deterministic startup and mode-transition sequencing.

A flight component contract SHALL be able to declare period, deadline, priority, criticality, maximum activations, stack budget, memory pool and overrun response.

21.5 Flight data plane

The innermost flight-control data plane SHALL use statically resolved direct calls, bounded queues, shared memory or deterministic bus adapters. Zenoh or other general distributed middleware MAY be used at payload, ground or non-critical boundaries but SHALL NOT be assumed suitable for the innermost critical loop.

The profile SHOULD support adapters for mission-selected links, including CAN, serial links, SpaceWire, MIL-STD-1553, deterministic Ethernet and hardware-specific buses. Support for a bus means implementing a controlled adapter and conformance tests; it does not imply qualification of every hardware interface.

21.6 Command and data handling

Neuradix Flight SHALL provide typed services for:

- command dispatch and verification;
- telemetry channels and packetization;
- events and event severity;
- mission parameters and tables;
- time correlation;
- stored command sequences;
- persistent state and file/data products;
- communication-link status;
- boot and reset reason;
- configuration and software version reporting.

Ground dictionaries SHALL be generated from the same contracts used by the flight application.

21.7 FDIR and redundancy

FDIR SHALL be represented as explicit, testable state machines and policies. Supported responses SHOULD include retry, component restart, partition reset, device isolation, sensor substitution, redundant-computer switch, degraded mode, safe mode and mission abort request.

The platform SHALL support:

- dual and triple instance patterns;
- state synchronization;

- value comparison and voting;
- disagreement detection;
- active/standby and hot/warm/cold redundancy;
- cross-strapped device assignment;
- fault-containment regions;
- detection of common-cause configuration or software faults.

Redundancy policy SHALL not imply independence. The assurance process must evaluate common software, design, compiler, hardware and requirements faults.

21.8 Space environment and hardware integrity

The profile SHALL expose hooks and telemetry for:

- ECC/EDAC status and memory scrubbing;
- watchdogs and processor resets;
- persistent-storage integrity and redundant copies;
- configuration checksums and transactional updates;
- bus and link error counters;
- radiation-event reporting where hardware provides it;
- thermal, power and clock anomalies;
- boot self-test and periodic built-in test.

Rust memory safety reduces classes of software defect but does not prevent radiation-induced corruption, hardware lockup, sensor failure or logic errors.

21.9 Boot, update and recovery

A flight deployment SHOULD support:

1. immutable or protected boot code;
2. verified boot of a signed image;
3. configuration and compatibility validation;
4. hardware self-test;
5. safe initialization mode;
6. explicit transition to mission-ready state;
7. A/B image banks where remote update is permitted;
8. interrupted-update recovery and rollback;
9. activation only under an authorised operational procedure.

Flight systems SHALL NOT pull arbitrary packages directly from a public registry.

21.10 Rust assurance profile

A mission SHALL define an approved Rust subset and toolchain. Neuradix tooling SHOULD enforce:

- approved compiler, linker and target versions;
- reproducible build containers;
- approved crates and feature flags;
- static analysis and lint configuration;
- controlled use of procedural macros and generated code;
- justification and test linkage for each unsafe block;
- stack, memory and timing analysis;
- source-to-object traceability where required;
- known compiler and dependency anomalies.

A qualified toolchain may reduce project effort, but no toolchain qualification automatically certifies a complete mission application.

21.11 Assurance and standards alignment

Neuradix Flight SHALL be designed to produce work products useful to NASA and ECSS processes, including requirements, architecture, interfaces, traceability, verification, configuration management and software product assurance. The platform SHALL avoid claiming blanket conformance because tailoring, criticality classification and mission approval remain project responsibilities.

The evidence kit SHOULD include:

- system and software requirements;
- interface control documents;
- hazard-linked safety requirements;
- requirements-to-code-to-test traceability;
- architecture and data-flow descriptions;
- FMEA/FMECA and fault-tree inputs;
- source-code and dependency standards;
- test procedures, results and coverage;
- timing, stack and memory analyses;
- compiler/tool validation evidence;
- configuration and release records;
- software bill of materials;
- known-problem and waiver records;
- cybersecurity and command-authority analysis.

21.12 Launcher-specific functions

A mature launcher deployment MAY include navigation, guidance, supervisory flight control, sequencing, staging coordination, telemetry and health management. The following requirements apply:

- engine, thrust-vector, staging, separation and abort commands SHALL have independent inhibits and positive authority validation;
- critical event sequences SHALL be deterministic, time-bounded and testable in lockstep simulation;
- local controllers SHALL enforce electrical and mechanical limits independent of the main planner;
- a safety or abort function SHALL not depend on Python, cloud connectivity or a general-purpose UI;
- command loss, reset, sensor disagreement and partial actuator failure SHALL have defined responses;
- mission time, event time and measurement time SHALL remain distinguishable;
- range or external safety interfaces SHALL be treated as independent trust boundaries.

21.13 Space simulation profile

Neuradix Sim SHOULD provide reusable models for:

- six-degree-of-freedom launch and spacecraft dynamics;
- atmospheric, orbital and gravity models;
- propulsion, actuator and mechanism dynamics;
- navigation sensors and timing errors;
- communication delay, outage and packet corruption;
- power, battery, thermal and radiation-event behaviour;
- staging, separation and deployment events;
- Monte Carlo dispersion and fault campaigns.

Model pedigree, version, validation status and applicable operating envelope SHALL be recorded with each result.

21.14 Flight Alpha acceptance criteria

The first Flight Alpha release SHALL be considered an engineering prototype, not a certified flight product. It SHOULD demonstrate:

- static topology generation;

- bounded message queues and memory pools;
- deterministic rate-group execution;
- command, telemetry, events and parameters;
- watchdog and safe-mode transition;
- Linux simulation plus one RTOS or bare-metal target;
- reproducible build and evidence bundle;
- HIL execution with a representative flight computer;
- a non-critical payload or laboratory avionics demonstration.

22. Hardware capability model

22.1 Capability-based drivers

Drivers SHOULD implement stable semantic capabilities rather than exposing only vendor-specific byte interfaces.

Initial capabilities:

- Camera;
- InertialSensor;
- DepthSensor;
- VelocityLog;
- PositionSource;
- Sonar;
- Thruster;
- MotorController;
- Battery;
- Manipulator;
- SafetyInput;
- Lighting;
- AcousticModem.

22.2 Capability requirements

Each hardware capability MUST define:

- commands and data outputs;
- units and frames;
- timing and timestamp source;
- health model;
- calibration model;
- supported rates and modes;
- command limits;
- error states;
- simulation equivalent;
- conformance tests.

22.3 Driver package grades

The package catalogue MAY publish compatibility grades:

- Experimental;
- Community Tested;
- Vendor Supported;
- Neuradix Certified;
- Safety Assessed for a named profile.

23. Simulation architecture

23.1 Simulation as a deployment target

Simulation MUST run the same application components and contracts used on physical hardware wherever practical.

23.2 Simulation services

Neuradix Sim SHOULD provide:

- deterministic scheduler integration;
- world and scenario loading;
- physics stepping;
- sensor simulation;
- actuator dynamics;
- environment models;
- scenario event scripting;
- fault injection;
- result assertions;
- Monte Carlo execution;
- HIL/SIL orchestration;
- recording to the same format as real missions.

23.3 Scenario format

```yaml
apiVersion: neuradix.io/v1alpha1
kind: Scenario
metadata:
  name: current-disturbance-return-home
spec:
  seed: 10427
  duration: 45min
  world: worlds/coastal-survey.nworld
  vehicle: explorer-auv-01
  environment:
    current:
      direction: 240deg
      speed: 0.8m/s
  faults:
    - at: 18min
      target: dvl
      action: lose_bottom_lock
      duration: 120s
  assertions:
```

```
  - expression: safety.maximum_depth_violations == 0
  - expression: mission.final_state == "recovered"
```

23.4 Marine simulation profile

The marine reference simulator SHOULD include:

- six-degree-of-freedom rigid-body dynamics;
- buoyancy and ballast;
- hydrodynamic damping and added mass;
- thruster curves, saturation and lag;
- currents and turbulence models;
- pressure/depth sensor models;
- IMU bias and drift;
- DVL bottom lock and water track;
- acoustic positioning;
- multibeam, profiling and imaging sonar;
- camera attenuation, backscatter and turbidity;
- acoustic modem delay, loss and bandwidth;
- energy and battery model.

23.5 Determinism

A deterministic scenario MUST record:

- random seeds;
- simulator version;
- component versions;
- topology and configuration hashes;
- initial world state;
- clock policy;
- external inputs.

23.6 Space and launcher simulation profile

The simulation framework SHOULD support launch, orbital and spacecraft models as defined in Section 21. Model metadata SHALL identify source, validation status, parameter set, applicable envelope and uncertainty. Space simulation SHOULD be compatible with the same command, telemetry and timing contracts used by Neuradix Ground and Flight.

24. Recording, replay and data management

24.1 Recording format

MCAP SHOULD be the primary external log container. Neuradix metadata, schemas and indexes MAY be stored in MCAP records or an associated signed manifest.

24.2 Recording scope

A mission recording SHOULD include:

- selected streams, state, commands, tasks and events;
- topology manifest;
- component hashes and versions;
- contract schemas;
- configuration snapshots;
- clock synchronization data;

- calibration references;
- operator commands;
- safety interventions;
- random seeds;
- hardware inventory;
- software bill of materials.

24.3 Recording policies

Per-stream policies MUST support:

- full recording;
- sampled recording;
- triggered recording;
- rolling buffer;
- metadata only;
- disabled;
- encryption;
- retention duration;
- priority under storage pressure.

24.4 Replay modes

- real-time;
- accelerated;
- lockstep;
- step-by-step;
- branch replay with component substitution;
- fault replay;
- counterfactual controller replay;
- partial graph replay.

24.5 Reproducibility objective

The platform SHOULD enable a field incident to be reconstructed locally with one command:

```
neuradix replay mission-2026-06-22.mcap --restore-deployment --lockstep
```

25. Observability and explainability

25.1 Telemetry model

Neuradix SHOULD use OpenTelemetry-compatible traces, metrics and logs, with Rust `tracing` integration.

25.2 Mandatory runtime metrics

- component CPU time;
- memory usage;
- GPU usage;
- queue depth;
- publication/subscription rate;
- payload throughput;
- end-to-end latency;
- deadline misses;
- dropped/superseded messages;

- network loss and reconnects;
- clock quality;
- restart count;
- storage pressure;
- safety interventions.

25.3 Causal lineage

Commands affecting actuation MUST support lineage linking:

- originating sensor samples/events;
- estimator outputs;
- planner/controller decision;
- authority decision;
- safety constraint result;
- final actuator command.

25.4 Explainability query

Studio and CLI SHOULD answer:

```
neuradix explain command propulsion/thrust --at 2026-06-22T14:32:18.420Z
```

The result SHOULD show the causal chain, configuration, software versions, timing and any safety modification.

25.5 Health model

Each component SHALL publish a structured health state:

- healthy;
- degraded;
- unhealthy;
- unavailable;
- unknown.

Health MUST include reasons, evidence, timestamp and recommended action.

26. Security architecture

26.1 Security principles

- least privilege;
- explicit component identity;
- signed artifacts;
- authenticated commands;
- encrypted transport where required;
- auditable changes;
- secure defaults;
- offline verification;
- rollback protection.

26.2 Component identity

Each component instance MUST have:

- package identity;
- package signature status;
- deployment identity;

- runtime instance identity;
- declared capabilities;
- cryptographic credentials appropriate to the profile.

26.3 Capability permissions

```
permissions:
  publish:
    - perception/detections
  subscribe:
    - camera/front
  query:
    - calibration/camera/front
  command: []
  actuatorAccess: false
  filesystem:
    read:
      - /models/object-detector
    write: []
  network:
    outbound: none
```

26.4 Package security

Packages MUST support:

- content hashes;
- digital signatures;
- publisher identity;
- dependency inventory;
- SBOM;
- build provenance;
- vulnerability advisory linkage;
- revocation status.

26.5 Update security

OTA updates SHOULD support:

- staged rollout;
- health-gated promotion;
- A/B or rollback-capable deployment;
- downgrade policy;
- signed metadata;
- interrupted-transfer recovery;
- offline update bundles;
- audit events.

26.6 Secret management

Secrets MUST NOT be embedded in ordinary manifests or logs. The runtime MUST support secret references and pluggable secret providers.

27. Packaging and registry

27.1 Package model

Neuradix SHOULD use OCI-compatible artifacts for distribution. A package may contain:

- native binaries;
- Python wheel/environment lock;
- WebAssembly component;
- contract schemas;
- default configuration;
- permissions;
- health model;
- documentation;
- tests;
- SBOM and provenance;
- architecture/platform variants.

27.2 Package manifest

```yaml
apiVersion: neuradix.io/v1alpha1
kind: Package
metadata:
  name: depth-controller
  namespace: io.neuradix.reference.auv
  version: 1.4.2
spec:
  runtime: rust-native
  entrypoint: bin/depth-controller
  targets:
    - linux-x86_64
    - linux-aarch64
  contracts:
    provides:
      - io.neuradix.control/vertical-thrust-request@1
    requires:
      - io.neuradix.navigation/vehicle-depth@1
  resources:
    cpu: 0.25
    memory: 32MiB
  execution:
    class: deterministic
  permissions:
    actuatorAccess: false
```

27.3 Registry functions

The registry SHOULD provide:

- immutable versioned artifacts;
- signature verification;
- package search;
- compatibility metadata;
- supported hardware/architecture matrix;

- vulnerability notices;
- deprecation status;
- conformance results;
- mirroring for offline sites.

28. Deployment and orchestration

28.1 Deployment manifest

A deployment describes nodes, components, placement, contracts, connections, resources, policies and safety configuration.

```yaml
apiVersion: neuradix.io/v1alpha1
kind: RobotDeployment
metadata:
  name: explorer-auv-01
spec:
  profile: marine-auv
  nodes:
    - name: control-computer
      target: linux-aarch64
      labels:
        role: vehicle-control
    - name: safety-mcu
      target: cortex-m7
      labels:
        role: safety
  components:
    - name: depth-controller
      package: registry.neuradix.example/reference/depth-controller:1.4.2
      node: control-computer
      execution:
        class: deterministic
        cpuAffinity: [2]
      restart: on-failure
    - name: object-detector
      package: registry.neuradix.example/reference/object-detector:2.1.0
      node: control-computer
      execution:
        class: ai
      resources:
        gpuMemory: 2GiB
  connections:
    - from: pressure-sensor.depth
      to: depth-controller.depth
    - from: depth-controller.thrust
      to: safety.vertical-thrust-request
```

28.2 Graph compiler

Before deployment, the compiler MUST validate:

- contract compatibility;

- schema versions;
- units and frames;
- clock domains;
- missing providers;
- cycles where prohibited;
- queue bounds;
- resource capacity;
- node architecture compatibility;
- permissions;
- actuator authority path;
- safety-policy presence;
- estimated network bandwidth;
- Python/AI components in deterministic paths;
- package signatures and revocation.

28.3 Deployment modes

- local development;
- single robot;
- multi-node robot;
- simulator;
- hardware-in-the-loop;
- fleet-managed;
- air-gapped/offline.

28.4 Production immutability

A production deployment MUST have a content-addressed manifest hash. Any runtime mutation MUST be recorded as a new signed revision or an explicitly permitted operational parameter change.

29. Fleet and offline operation

29.1 Offline-first rules

A robot MUST NOT depend on a cloud service for:

- basic boot;
- safety;
- local control;
- active mission execution;
- local logging;
- fail-safe behaviour.

29.2 Link policies

```
stream: sonar/raw
policy:
  localRecord: full
  remoteTransfer: disabled

stream: vehicle/health
policy:
  remoteTransfer:
    priority: critical
```

```
    maximumRate: 1Hz
    delivery: reliable

stream: mission/events
policy:
  remoteTransfer:
    priority: high
    storeAndForward: true
```

29.3 Fleet functions

Neuradix Fleet MAY provide:

- inventory;
- health summaries;
- mission distribution;
- configuration rollout;
- update orchestration;
- log selection and upload;
- geospatial overview;
- remote support sessions;
- audit and access control;
- fleet-wide policy enforcement.

29.4 Constrained communication

The marine profile SHOULD support:

- message prioritization;
- compact health summaries;
- store-and-forward;
- resumable transfer;
- acoustic modem gateways;
- command expiry;
- duplicate suppression;
- delayed acknowledgement;
- link-cost-aware routing.

30. AI and machine-learning support

30.1 AI principles

AI outputs are estimates, not privileged truth. AI components MUST remain subject to confidence handling, fallback logic, authority and safety constraints.

30.2 Standard AI types

- images and video;
- tensors;
- point clouds;
- detections;
- segmentations;
- tracks;
- embeddings;
- model confidence;

- uncertainty;
- model provenance.

30.3 Model packages

A model package SHOULD include:

- model artifact;
- runtime requirements;
- preprocessing/postprocessing definition;
- expected input contract;
- output contract;
- performance profile;
- training-data provenance reference;
- evaluation results;
- model card;
- license;
- signature.

30.4 Deployment patterns

- active inference;
- shadow inference;
- A/B comparison;
- canary rollout;
- fallback algorithm;
- confidence-gated publication;
- dataset capture trigger;
- drift monitoring.

30.5 Accelerator scheduling

GPU/accelerator workloads SHOULD declare:

- memory requirement;
- latency target;
- batch policy;
- priority;
- exclusivity requirement;
- supported devices;
- fallback mode.

31. Interoperability

31.1 Bridge architecture

A bridge is a managed component that translates external protocols to Neuradix contracts. It MUST expose conversion, timing and data-loss behaviour.

31.2 Initial bridges

Priority 1:

- ROS 2 topics, services and actions;
- MAVLink/MAVSDK and ArduPilot/ArduSub;
- CAN and serial;

- WebSocket and HTTP/gRPC for operator systems.

Priority 2:

- DDS-native systems;
- OPC UA;
- SpaceWire and mission-specific packet gateways;
- CCSDS-compatible ground/flight packet adapters where selected by a mission;
- MQTT;
- CANopen;
- MOOS-IvP;
- Goby.

31.3 ROS 2 bridge principles

- map ROS message schemas to explicit Neuradix contracts;
- preserve source timestamps where available;
- expose ROS QoS conversion;
- warn about unsupported semantics;
- isolate DDS discovery from the internal graph;
- support selected packages without requiring ROS 2 inside every deployment.

32. Neuradix Studio

32.1 Product form

Studio SHOULD be local-first and browser-based, with an optional desktop wrapper. It MUST operate without internet access.

32.2 Core views

- live component graph;
- deployment editor;
- contract inspector;
- stream/state browser;
- latency and bandwidth heat map;
- logs, metrics and traces;
- command lineage viewer;
- time-series plots;
- image/video viewer;
- point-cloud and map viewer;
- coordinate-frame viewer;
- mission/scenario editor;
- recording and replay controls;
- fault injection panel;
- package and driver manager;
- security and permission view;
- update and deployment status.

32.3 Studio safety rules

- actuator commands require explicit authenticated permission;
- dangerous commands require confirmation or configured multi-step approval;
- production changes show the exact resulting deployment revision;
- all operator actions are audited;
- Studio disconnection does not compromise robot operation.

33. Command-line interface

The CLI SHALL use consistent resource-oriented commands.

```
neuradix new component depth-controller --language rust
neuradix new component object-detector --language python
neuradix contract check contracts/
neuradix build
neuradix test
neuradix graph validate robot.yaml
neuradix sim run scenarios/dive.yaml
neuradix inspect stream navigation/depth
neuradix record start --profile mission
neuradix replay mission.mcap --lockstep
neuradix explain command propulsion/thrust --at <timestamp>
neuradix package sign ./target/package
neuradix deploy robot.yaml --target explorer-auv-01
neuradix doctor
```

CLI output MUST be available in human-readable and machine-readable JSON forms.

34. Testing and verification

34.1 Test layers

- unit tests;
- contract tests;
- component harness tests;
- graph integration tests;
- deterministic replay tests;
- simulation scenarios;
- property-based tests;
- fuzzing;
- fault-injection tests;
- hardware conformance tests;
- HIL tests;
- performance and jitter tests;
- security tests.

34.2 Contract test generation

The contract compiler SHOULD generate:

- valid and invalid sample data;
- unit/frame compatibility tests;
- schema evolution tests;
- queue-overflow tests;
- timeout/deadline tests;
- capability conformance stubs.

34.3 Fault injection

Fault injection SHOULD include:

- message loss;

- duplication;
- reordering;
- delay and jitter;
- stale timestamps;
- clock drift;
- process crash;
- CPU starvation;
- memory pressure;
- network partition;
- sensor freeze;
- sensor bias/drift;
- corrupted data;
- actuator saturation;
- battery degradation.

34.4 Reproducibility

CI scenario failures MUST output a replayable artifact containing the seed, topology, configuration, software versions and relevant recorded data.

34.5 Qualification-oriented evidence

For Flight and Safety profiles, the test system SHOULD generate requirements traceability, target/tool versions, test procedure identity, coverage, timing results, resource-watermark data and signed result bundles. Evidence generation SHALL be reproducible and SHALL preserve failed as well as passed results.

35. Performance and quality targets

These are design targets to be validated by published benchmarks, not guarantees for all hardware.

35.1 Runtime targets

- No unbounded queue in a production-validated graph.
- Zero-copy handoff for supported local large-buffer paths.
- Component start and health state available within defined deployment profile budget.
- Deterministic executor jitter reported with percentile distribution.
- End-to-end latency attributable by component and transport stage.
- Runtime overhead benchmarked separately from application computation.

35.2 Reference benchmark suite

The project SHOULD publish reproducible benchmarks for:

- in-process small messages;
- shared-memory images and point clouds;
- local network streams;
- reconnect after link interruption;
- command round-trip;
- deterministic periodic scheduling;
- MCAP recording under storage pressure;
- Python zero-copy access;
- graph startup;
- failover and restart;
- embedded endpoint throughput.

35.3 Quality gates

Core releases SHOULD require:

- supported-platform CI;
- dependency/license audit;
- fuzzing corpus execution;
- sanitizer/Miri checks where applicable;
- reproducible benchmark comparison;
- compatibility test suite;
- signed release artifacts;
- SBOM generation;
- documented migration notes.

36. Repository architecture

Recommended initial organisation:

```
neuradix/
  Cargo.toml
  crates/
    runtime/
    sdk/
    contracts/
    graph/
    transport-api/
    transport-local/
    transport-shm/
    transport-zenoh/
    time/
    frames/
    safety/
    record/
    telemetry/
    package/
    cli/
    testkit/
  python/
    neuradix/
    examples/
  embedded/
  flight/
  ground/
  safety/
  assurance/
    core/
    rtic/
    embassy/
  studio/
    frontend/
    backend/
```

```
bridges/
  ros2/
  mavlink/
  can/
simulation/
  core/
  marine/
contracts/
  standard/
  marine/
examples/
  minimal-robot/
  reference-auv/
docs/
rfcs/
governance/
```

A monorepo is recommended during early architectural development to keep contracts and releases coherent. Independent repositories may be split later when boundaries are stable.

37. API stability and governance

37.1 Stability levels

- Experimental;
- Preview;
- Stable;
- Long-Term Support;
- Deprecated;
- Removed.

Stable contracts MUST have documented compatibility guarantees.

37.2 RFC process

Material architectural changes SHOULD use public RFCs containing:

- problem statement;
- requirements;
- proposed design;
- alternatives;
- compatibility impact;
- security/safety impact;
- migration plan;
- test plan.

37.3 Governance recommendation

Initial governance MAY remain founder-led, but the project SHOULD publish:

- contribution guide;
- code of conduct;
- security policy;
- release process;
- decision/RFC process;

- maintainer roles;
- conflict-of-interest policy;
- roadmap.

37.4 Licensing

Recommended model:

- Apache License 2.0 for runtime, SDKs, contracts, CLI, core Studio and official bridges;
- permissively licensed examples and standard contracts;
- trademark policy controlling the Neuradix name and certification marks;
- commercial hosted services, support, certification and validated distributions without withholding essential core APIs.

This preserves genuine open-source adoption while enabling revenue through engineering services, support, hosted fleet services, certified packages and target-specific validation.

38. Initial reference AUV

38.1 Purpose

The reference AUV is both a demonstrator and a conformance target. It should prove the entire lifecycle from simulation to deployment and replay.

38.2 Reference subsystems

- Linux vehicle computer;
- safety/IO MCU;
- IMU;
- pressure sensor;
- DVL;
- GNSS when surfaced;
- imaging sonar;
- forward camera;
- acoustic modem;
- battery monitor;
- thruster controller;
- mission planner;
- state estimator;
- depth/heading/speed control;
- safety manager;
- recorder;
- operator station.

38.3 Demonstration mission

1. Load signed mission and deployment.
2. Run pre-dive health and authority checks.
3. Dive to survey depth.
4. Execute lawnmower survey.
5. Detect and avoid an obstacle.
6. Simulate DVL bottom-lock loss.
7. Continue in degraded navigation mode.
8. Simulate communication loss.
9. Complete or abort according to policy.
10. Surface and transfer prioritized logs.

11. Replay the mission and explain a selected actuator command.
12. Replace one controller and run a counterfactual replay.

38.4 Secondary reference space demonstrator

After the marine vertical slice is stable, Neuradix SHOULD add a laboratory or non-critical flight demonstrator comprising a simulated launch/spacecraft model, a Flight Alpha runtime, Ground command/telemetry, an independent safety monitor and HIL hardware. This demonstrator validates profile separation without displacing the AUV as the first complete robotics reference.

39. Delivery roadmap

Phase 0 - Architecture validation

- approve RFC-0001 Component and Contract Model;
- validate Rust/Python zero-copy and process isolation;
- prove transport independence with in-process, shared memory and Zenoh;
- define profile policy schema for Edge, Embedded, Flight, Safety, Sim and Ground;
- demonstrate deterministic recording and replay;
- establish performance baselines.

Phase 1 - Minimum viable platform

- Runtime, Contracts and Data Plane;
- Rust SDK and Python worker SDK;
- topology compiler and local supervisor;
- Neuradix Record and a minimal Studio;
- Edge reference application;
- basic ROS 2 and MAVLink bridges.

Phase 2 - Dependability release

- authority leases and safety constraints;
- health supervision and FDIR primitives;
- security identity and signed packages;
- static production topology;
- deterministic replay and causal command lineage;
- Embedded profile on at least one MCU family.

Phase 3 - Marine reference platform

- complete AUV/USV vertical slice;
- sonar, camera, IMU, DVL and pressure capabilities;
- hydrodynamic simulation and HIL;
- constrained-link and offline mission operation;
- field trials and public benchmark/evidence data.

Phase 4 - Space simulation and Ground

- launch and spacecraft simulation models;
- Ground command, telemetry, timeline and procedure services;
- Flight contract generation and static topology;
- HIL with representative avionics;
- assurance evidence bundle generation.

Phase 5 - Flight Alpha and payload demonstration

- restricted Rust runtime;

- command/data handling, time, watchdog and FDIR;
- one RTOS or bare-metal target;
- isolated payload or laboratory flight demonstration;
- independent review and published limitations.

Phase 6 - Ecosystem, Fleet and mission heritage

- multi-robot and constellation operations;
- package approval channels and long-term support releases;
- additional hardware and protocol partners;
- progressively higher-criticality missions only after sufficient assurance and successful operational history.

40. Prioritised backlog

P0 - Essential foundation

- contract schema and compiler;
- runtime lifecycle;
- Rust SDK;
- Python process SDK;
- local and shared-memory transport;
- Zenoh adapter;
- bounded queues;
- time domains;
- deployment graph validation;
- MCAP record/replay;
- CLI and testkit.

P1 - Differentiators

- semantic units and frames;
- deterministic executor;
- authority leases;
- safety constraints;
- causal tracing;
- command explanation;
- deterministic simulation clock;
- signed packages;
- ROS 2/MAVLink bridges;
- Studio graph and timeline.

P2 - Space and assurance expansion

- Flight static runtime and rate groups;
- Ground command and telemetry dictionaries;
- RTOS/bare-metal target;
- FDIR state-machine tooling;
- requirements and verification evidence generator;
- space digital-twin model package;
- HIL avionics reference rig.

P3 - Ecosystem scale

- embedded profile;
- WebAssembly plugins;

- registry;
- fleet management;
- model registry;
- certification/conformance portal;
- advanced distributed scheduling;
- redundant component voting.

41. Acceptance criteria for version 1.0

Version 1.0 acceptance applies to the common platform and Edge/Sim/Record/Studio foundations. It does not constitute flight or human-rating certification. Flight Alpha has separate criteria in Section 21.14.

Neuradix 1.0 SHALL not be declared until all of the following are satisfied:

1. Stable Rust and Python SDKs are published.
2. A versioned contract and compatibility policy is documented.
3. Production graphs reject unbounded queues unless explicitly waived.
4. The reference robot runs the same application contracts in simulation and hardware.
5. MCAP recording and deterministic/lockstep replay are supported.
6. A Python component can crash without terminating control and safety processes.
7. The platform exposes units, frames and clock domains.
8. Authority and safety paths protect all reference actuators.
9. Signed package and deployment verification is available.
10. ROS 2 and MAVLink bridges are functional.
11. Studio can inspect the graph, data, health, timing and command lineage.
12. Published benchmark procedures exist for latency, throughput, jitter and recording.
13. Security policy, SBOM and vulnerability process are published.
14. Upgrade and rollback procedures are tested.
15. The complete reference AUV scenario is reproducible from public instructions.

42. Key risks and mitigations

Risk	Impact	Mitigation
Attempting to match the ROS ecosystem too early	scope collapse	focus on platform fundamentals and bridges
Building a new transport unnecessarily	long delay and protocol defects	backend-neutral API; use Zenoh initially
Claiming real-time guarantees too broadly	unsafe expectations	explicit execution classes and validated profiles
Python performance or crash impact	control instability	process isolation, bounded inputs, no deterministic-path use
Contract system becomes too complex	developer rejection	progressive disclosure, good defaults, generated code
Studio consumes engineering capacity before runtime stabilises	delayed core	implement inspection first, visual authoring later
Weak driver ecosystem	adoption barrier	bridge support, capability contracts, vendor conformance
Over-centralised governance	community hesitation	public RFCs, transparent roadmap, Apache-2.0 core
Security added too late	redesign and field exposure	identity, permissions and signed artifacts from early releases
Deterministic replay is incomplete	lost differentiator	define time/random/config capture in Phase 0

Risk	Impact	Mitigation
Name confusion with other neuro/robotics brands	commercial/legal risk	use full Neuradix mark consistently and obtain trademark advice

43. Decisions recommended now

The following decisions should be made immediately:

1. Adopt **Neuradix Robotics Platform** as the formal product name.
2. Use **Neuradix** as the sole master brand.
3. Use Rust for the trusted core and Python as an isolated first-class extension environment.
4. Use a transport-neutral API with Zenoh as the first network backend.
5. Use MCAP for primary recording and replay storage.
6. Use OpenTelemetry-compatible observability.
7. Use OCI artifacts for packaging and distribution.
8. Use Apache License 2.0 for the core.
9. Build the reference AUV and simulator as the first complete vertical slice.
10. Start with architecture RFCs and a Phase 0 proof of concept before building a polished Studio.

44. First 90-day engineering plan

Weeks 1-2: foundation decisions

- create project charter and governance files;
- create monorepo;
- define terminology;
- write RFC-0001 component model;
- write RFC-0002 contract model;
- write RFC-0003 execution classes;
- write RFC-0004 transport abstraction;
- write RFC-0005 recording/replay model;
- define reference AUV minimum topology.

Weeks 3-5: minimum runtime

- implement component identity and lifecycle;
- implement local Stream and State primitives;
- implement bounded queues;
- implement a simple graph manifest;
- generate Rust types from one contract;
- publish structured lifecycle and health events.

Weeks 6-8: language and transport vertical slice

- add Python SDK through PyO3/Maturin;
- add managed Python process supervision;
- add Zenoh transport adapter;
- add shared-memory image buffer prototype;
- demonstrate Rust camera producer and Python detector consumer.

Weeks 9-10: record and reproduce

- record streams, state and events to MCAP;
- capture topology/configuration hashes;

- add replay clock;
- demonstrate repeatable output from a fixed-seed component.

Weeks 11-12: reference demo

- create minimal AUV depth-control simulator;
- implement depth sensor, controller, safety limiter and thruster model;
- display graph and signals in a minimal Studio page;
- demonstrate Python process crash isolation;
- publish benchmark and architecture report;
- decide whether the foundations satisfy Phase 0 exit criteria;
- run a paper architecture review of the Flight and independent safety-island profiles without attempting critical flight implementation.

45. Technical reference rationale

The recommended foundations are selected because they provide existing, actively maintained building blocks while allowing Neuradix to preserve a transport-independent architecture:

- Zenoh supplies publish/subscribe and queryable abstractions and has a Rust implementation.
- PyO3 supports native Python modules implemented in Rust and embedding Python from Rust; Maturin supports packaging those bindings.
- MCAP is an open container format for timestamped multimodal data with Rust and Python libraries.
- OpenTelemetry provides vendor-neutral traces, metrics and logs.
- WIT defines contracts for WebAssembly Component Model interfaces.
- RTIC and Embassy provide Rust-oriented embedded execution models.
- OCI specifications provide a standard image/artifact packaging foundation.
- F Prime and cFS provide useful flight-framework reference patterns for typed components, command/telemetry services and reusable flight applications.
- RTEMS provides an open RTOS path with spaceflight use.
- NASA and ECSS software and assurance standards inform the evidence-oriented Flight profile, without implying blanket compliance.

References

1. Zenoh documentation: https://zenoh.io/docs/
2. Zenoh abstractions: https://zenoh.io/docs/manual/abstractions/
3. PyO3 user guide: https://pyo3.rs/
4. PyO3 Python typing guidance: https://pyo3.rs/main/python-typing-hints
5. MCAP: https://mcap.dev/
6. MCAP API reference: https://mcap.dev/reference
7. OpenTelemetry documentation: https://opentelemetry.io/docs/
8. OpenTelemetry specification: https://opentelemetry.io/docs/specs/otel/
9. WebAssembly Component Model and WIT: https://component-model.bytecodealliance.org/
10. RTIC documentation: https://rtic.rs/
11. Embassy documentation: https://embassy.dev/
12. Open Container Initiative: https://opencontainers.org/
13. NASA F Prime documentation: https://fprime.jpl.nasa.gov/latest/
14. NASA Core Flight System repository: https://github.com/nasa/cFS
15. RTEMS project: https://www.rtems.org/
16. NASA Software Engineering Handbook, NASA-HDBK-2203: https://standards.nasa.gov/standard/NASA/NASA-HDBK-2203
17. NASA Software Assurance and Software Safety Standard, NASA-STD-8739.8B: https://standards.nasa.gov/standard/NASA/NASA-STD-87398
18. ECSS-E-ST-40C Rev.1, Software, 30 April 2025: https://ecss.nl/standard/ecss-e-st-40c-rev-1-software-30-april-2025/
19. ECSS-Q-ST-80C Rev.2, Software product assurance, 30 April 2025: https://ecss.nl/standard/ecss-q-st-80c-rev-2-software-product-assurance-30-april-2025/
20. ECSS-E-ST-40-07C Rev.1, Simulation modelling platform Level 1, 5 August 2025: https://ecss.nl/standard/ecss-e-st-40-07c-rev-1-simulation-modelling-platform-level-1-5-august-2025/
21. Ferrocene qualified Rust toolchain documentation: https://ferrocene.dev/